CS336 作业 2：系统——训练与并行（Systems）

原文：Stanford CS336 作业说明（英文原版）

本文档为英文原版作业说明的非官方中文翻译，仅供学习参考；如与原文有歧义，请以英文原文为准。

目录

1 作业概览 **3**

2 性能分析与基准测试 **4**

2.1 性能分析 4

2.1.1 准备工作：导入你的 basics Transformer 模型 4

2.1.2 模型规模 5

2.1.3 端到端基准测试 5

2.1.4 Nsight Systems 性能分析器 6

2.1.5 混合精度 9

2.1.6 内存性能分析 11

3 单 GPU 内存 **12**

3.1 自动求导残差 13

3.1.1 算子融合 14

3.2 激活检查点 15

3.2.1 重计算 16

4 GPU 核函数 **18**

4.1 用 FlashAttention-2 优化注意力 18

4.1.1 PyTorch 注意力基准测试 18

4.2 JIT 编译注意力的基准测试 18

4.2.1 示例：加权和 19

4.2.2 FlashAttention-2 前向传播 25

4.2.3 可选：Triton 反向传播 32

5 分布式数据并行训练 33

5.1 PyTorch 单节点分布式通信 33

5.1.1 分布式应用基准测试的最佳实践 35

5.2 分布式数据并行训练的朴素实现 36

5.3 改进最小 DDP 实现 37

5.3.1 减少通信调用次数 37

5.3.2 让计算与单个参数梯度的通信重叠 38

6 优化器状态分片 40

7 完全分片数据并行 41

8 分析并行策略 43

8.1 通信原语 43

8.2 分析数据并行 45

8.3 分析完全分片数据并行 46

8.4 分析张量并行 47

8.5 二维并行（FSDP + TP） 49

9 排行榜 50

1 作业概览

在本次作业中，你将通过动手实践，学习如何提升单 GPU 上的训练速度，以及如何把训练扩展到多块 GPU 上。

你要实现的内容

1. 基准测试与性能分析框架（benchmarking and profiling harness）
2. 激活检查点（activation checkpointing）
3. FlashAttention 2 的 Triton 核函数（kernel）
4. 分布式数据并行训练
5. 优化器状态分片
6. 完全分片数据并行（fully sharded data parallel）训练

代码的结构

作业代码与这份说明都发布在 GitHub 上：

github.com/stanford-cs336/assignment2-systems

请使用 Git 克隆该仓库。如果有任何更新，我们会通知你，届时你可以通过 `git pull` 获取最新版本。

1. `cs336-basics/` ：在本次作业中，你将对作业 1 中构建的一些组件进行性能分析。该文件夹包含作业 1 的助教参考答案代码，因此你可以在这里找到 `cs336-basics/pyproject.toml` 以及 `cs336-basics/cs336_basics` 包。如果你想使用自己的模型实现，可以修改根目录下的 `pyproject.toml` 文件，使其指向你自己的包。
2. `/` ：`cs336-systems` 的根目录。我们在其中创建了一个名为 `cs336_systems` 的空模块。注意这里没有任何代码，所以你完全可以一切从零开始、自行发挥。
3. `tests/*.py` ：该目录包含你必须通过的全部测试。这些测试会调用 `tests/adapters.py` 中定义的钩子（hook）。你需要实现这些适配器（adapter），把你的代码与测试连接起来。多写一些测试和/或修改测试代码对调试很有帮助，但你的实现必须能通过最初提供的测试套件。
4. `README.md` ：该文件包含关于预期目录结构的更多细节，以及一些配置环境的基本说明。

如何提交

你需要向 Gradescope 提交以下文件：

- `writeup.pdf` ：回答所有笔头问题。请对你的回答进行排版（typeset）。
- `code.zip` ：包含你编写的全部代码。

运行 `test_and_make_submission.sh` 中的脚本即可生成 `code.zip` 文件。

2 性能分析与基准测试

在作业的第一部分，我们将探索如何优化 Transformer 模型的性能，以最充分地利用 GPU。我们会先对模型进行性能分析（profiling），了解它在一次前向传播和反向传播过程中把时间和内存花在了哪里；然后用自定义 GPU 核函数（kernel）优化自注意力（self-attention）运算，使其比常规 PyTorch 实现跑得更快。在作业的后续部分，我们将利用多块 GPU，学习如何在集群上训练模型。

2.1 性能分析

在实现任何优化之前，最好先对程序做性能分析，弄清楚它把资源（例如时间和内存）花在了哪里。否则，我们可能会去优化那些在时间或内存上并不占大头的部分，从而看不到可衡量的端到端提升。

我们将实现三条性能评估路径：

1. 使用 Python 标准库进行简单的端到端基准测试，为前向和反向传播计时；
2. 使用 NVIDIA Nsight Systems 工具进行计算性能分析，了解这些时间如何分布在 CPU 和 GPU 的各个运算上；
3. 内存性能分析。

2.1.1 准备工作：导入你的 basics Transformer 模型

首先，确认你可以加载上一次作业的模型。在上一次作业中，我们把模型组织成了一个 Python 包，以便后续方便地导入。我们在 `./cs336-basics` 文件夹中加入了模型的参考实现，并在 `pyproject.toml` 文件中指向了它。像往常一样调用 `uv run [command]` 时，uv 会自动找到这个本地的 `cs336-basics` 包。如果你想使用自己的模型实现，可以修改 `pyproject.toml` 文件，让它指向你自己的包。

你可以用如下方式测试能否导入你的模型：

```
$ uv run python
Using CPython 3.13.13
Creating virtual environment at: /path/to/uv/env/dir
        Built cs336-systems @ file:///path/to/systems/dir
        Built cs336-basics @ file:///path/to/basics/dir
Installed 78 packages in 168ms
Python 3.13.13 (main, Apr  7 2026, 20:49:46) [Clang 22.1.1 ] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import cs336_basics
...
```

作业 1 的相关模块现在应该都可以使用了（例如，对于 `model.py`，可以用 `import cs336_basics.model` 导入）。

2.1.2 模型规模

在整个作业中，我们会对不同规模的模型进行基准测试和性能分析，以更好地理解它们的性能。为了感受规模变化带来的影响，我们将使用并引用下面这些模型配置。除排行榜（leaderboard）中的模型外，其余所有模型我们都使用 10000 的词表大小和 4 的批量大小（batch size），上下文长度则各不相同。本次作业（以及后续作业）需要以表格和图表的形式呈现大量结果。我们强烈建议你用代码自动生成 writeup 中的表格，因为在 LaTeX 或 Typst 中手工排版表格非常繁琐。可以参考 `pandas.DataFrame.to_latex()` 和 `pandas.DataFrame.to_typst()`，或者自己编写一个函数，从你偏好的表格表示生成它们。

规模	d_model	d_ff	num_layers	num_heads
small	768	3072	12	12
medium	1024	4096	24	16
large	1280	5120	36	20
xl	2560	10240	32	32
10B	4608	12288	50	36

表 1: 不同模型规模的规格说明。这些配置大多基于 GPT-2 的配置。

除非另有说明，否则使用 512 的上下文长度。

2.1.3 端到端基准测试

现在我们来编写一个简单的性能评估脚本。我们要测试模型的许多变体（改变精度、替换层等等），因此让你的脚本能够**通过命令行参数启用这些变体**会非常值得，方便日后运行。

作为起步，我们先对模型做最简单的性能分析：为前向传播、反向传播和优化器步骤计时。由于我们只测量速度和内存，使用随机的权重和数据即可。

性能测量很微妙——一些常见的陷阱会导致我们测到的并不是想要的量。对 GPU 代码做基准测试时，一个需要注意的地方是 CUDA 调用是异步的。当你调用一个 CUDA 核函数（例如调用 `torch.matmul`）时，PyTorch 的函数调用会立即把控制权返回给你的代码，而不会等待矩阵乘法完成。这样，CPU 可以继续抢先运行并调度新的运算，而 GPU 则在一旁完成矩阵乘法，这是一个重要的性能优势。另一方面，这也意味着，天真地测量 `torch.matmul` 调用返回所需的时间，并不能告诉我们 GPU 实际执行矩阵乘法花了多久。在 PyTorch 中，我们可以调用 `torch.cuda.synchronize()` 来等待所有已调度的 GPU 核函数执行完毕，从而更准确地测量 CUDA 核函数的运行时间。这里的同步指的是让 CPU 运行时与 GPU 运行时同步。记住这一点，下面我们来搭建基本的性能分析基础设施。

问题（`benchmarking_script`）：基准测试脚本（4 分）

(a) 编写一个脚本，对模型的前向传播、反向传播和优化器步骤进行基本的端到端基准测试。具体而言，你的脚本应支持以下功能：

- 根据给定的超参数（例如层数），初始化一个模型。
- 生成一个随机批次的数据。
- 先运行 w 个预热（warm-up）步骤（在开始计时之前），然后为 n 个步骤的执行计时（根据某个参数的取值，可以只做前向、做前向加反向，或者做前向、反向再加优化器步骤）。计时可以使用 Python 的 `timeit` 模块（例如使用 `timeit` 函数，或使用 `timeit.default_timer()`，它给出系统分辨率最高的时钟，因而比 `time.time()` 更适合作为基准测试的默认选择）。
- 在每个步骤之后调用 `torch.cuda.synchronize()`。

交付物：一个脚本：用给定的超参数初始化一个 basics Transformer 模型，创建一个随机批次的数据，并为仅前向、前向加反向、以及包含优化器步骤的完整训练步骤计时。

(b) 为第 2.1.2 节描述的各模型规模，测量前向、反向和优化器步骤的耗时。使用 5 个预热步骤，并在 10 个测量步骤上计算耗时的平均值和标准差。一次前向传播需要多长时间？反向传播呢？各次测量之间波动很大，还是标准差很小？

交付物：用 1-2 句话给出你的计时结果。

(c) 基准测试的一个注意事项是要执行预热步骤。在不做预热步骤的情况下重复你的分析。这会如何影响你的结果？你认为为什么会这样？再用 1 或 2 个预热步骤运行脚本试试。为什么结果可能仍然不同？

交付物：用 2-3 句话回答。

2.1.4 Nsight Systems 性能分析器

端到端基准测试无法告诉我们模型在前向和反向传播期间把时间和内存花在了哪里，因而也揭示不出具体的优化机会。要想知道程序在每个组件（例如函数）上花了多少时间，可以使用性能分析器（profiler）。执行分析器通过在函数开始和结束运行处插入探针（guard）对代码插桩，因此可以给出函数级别的详细执行统计信息（例如调用次数、平均耗时、在该函数上累计花费的时间等等）。

标准的 Python 分析器（例如 CProfile）无法分析 CUDA 核函数，因为这些核函数是在 GPU 上异步执行的。幸运的是，NVIDIA 自带一个分析器，可以通过命令行工具 `nsys` 使用。我们建议你通过包管理器获取最新版本，或使用其下载页面提供的安装程序。在作业的这一部分，你将使用 `nsys` 来分析你的 Transformer 模型的运行时行为。

使用 `nsys` 很简单：在前一节的 Python 脚本前面加上 `nsys profile` 运行即可。例如，可以用下面的命令对脚本 `benchmark.py` 做一次基本的分析：

```
$ uv run nsys profile -- python benchmark.py
```

然后你可以在本地机器上用 NVIDIA Nsight Systems 桌面应用程序查看分析结果。在分析的 `CUDA API` 行中选中某个特定的 CUDA API 调用（CPU 侧），就会在 `CUDA HW` 行中高亮显示所有相应的核函数执行（GPU 侧）。

一次更全面的分析运行可能如下：

```
$ uv run nsys profile --trace=cuda,cudnn,cublas,osrt,nvtx --pytorch=functions-trace,autograd-
    shapes-nvtx --cudabacktrace=all --python-backtrace=cuda --gpu-metrics-devices=0 -- python
    benchmark.py
```



图 1: 一份详细的 Nsight Systems 分析轨迹（trace）

在这个例子中，`--trace` 指定要记录哪些 API；`--pytorch` 在模块调用和自动求导期间插入 nvtx 标签；`--cudabacktrace` 和 `--python-backtrace` 提供更好的回溯信息，帮助你了解某个核函数是从代码中的哪里被调用的；`--gpu-metrics-devices` 指定要测量哪块 GPU 的利用率。

为运行添加分析并非没有代价，总体上会拖慢你的运行。通常更可取的做法是只在某次运行中启用你正关注的功能。具体来说，当不需要回溯信息时，你可能想移除 `--cudabacktrace=all` 和 `--python-backtrace=cuda`，因为它们的开销特别大。

我们鼓励你尝试 `nsys profile` 的各种命令行选项，感受一下它能做什么。你还可以用 NVTX 区间（range）标注你的代码，它们会以块的形式出现在分析的 NVTX 行中，涵盖所有 CUDA API 调用及相关的核函数执行。特别地，你应当使用 NVTX 区间来**忽略基准测试脚本中的预热步骤**（通过对分析中的 nvtx 标签施加 `--nvtx-capture` 过滤器）。你还可以区分出哪些核函数负责模型的前向和反向传播，甚至可以通过如下方式标注你的实现，区分出自注意力层的不同部分分别由哪些核函数负责：

```
...
import torch.cuda.nvtx as nvtx

@nvtx.range("scaled dot product attention")
def annotated_scaled_dot_product_attention(
    ... # Q, K, V, mask
):
```

```
    ...
    with nvtx.range("computing attention scores"):
        ... # compute attention scores between Q and K

    with nvtx.range("computing softmax"):
        ... # compute softmax of attention scores

    with nvtx.range("final matmul"):
        ... # compute output projection

    return ...
```

在基准测试脚本中，你可以通过下面的方式用带标注的版本替换原来的实现：

```
cs336_basics.model.scaled_dot_product_attention = annotated_scaled_dot_product_attention
```

最后值得注意的是，`torch.compile` 会让时间和资源难以归因到代码的特定部分。你可能需要用 `torch.compile` 和 nvtx 标注包裹或剥离代码的各个部分，才能把时间和资源使用正确地归因到源代码的各个部分。

问题（`nsys_profile`）：Nsight Systems 性能分析（5 分）

使用 `nsys`，从表 1 中自选两个模型规模，以及三个大于 128 的 2 的幂次上下文长度，对你的前向传播、反向传播和优化器步骤进行性能分析，其中可用的最大规模应当是内存能容纳的最长上下文长度。请选择你认为最有意思的组合。对每个分析结果回答以下问题：

(a) 你的前向传播总共花了多少时间？这与之前用 Python 标准库测得的结果是否一致？

交付物：用 1-2 句话回答。

(b) 在前向传播期间，哪个 CUDA 核函数占用最多的累计 GPU 时间？在模型的一次前向传播中，这个核函数被调用了多少次？当你同时做前向和反向传播时，占用运行时间最多的也是同一个核函数吗？（提示：查看“Stats System View”下的“CUDA GPU Kernel Summary”，并利用 NVTX 区间过滤，以确定模型的哪些部分对应哪些核函数。）

交付物：用 1-2 句话回答。

(c) 尽管绝大多数 FLOPs 都发生在矩阵乘法中，你会注意到还有几个其他核函数也占整体运行时间的相当一部分。在前向传播中，除矩阵乘法外，你还看到哪些核函数占了不可忽视的 CUDA 运行时间？

交付物：用 1-2 句话回答。

(d) 用你实现的 AdamW 分析一个完整训练步骤的运行（即前向传播、计算损失并运行一次反向传播，最后执行一次优化器步骤，就像训练时那样）。与推理（仅前向传播）相比，花在矩阵乘法上的时间比例有何变化？其他核函数呢？

交付物：用 1-2 句话回答。

(e) 在前向传播过程中，比较模型自注意力层内 softmax 运算与各矩阵乘法运算的运行时间。运行时间上的差异与 FLOPs 上的差异相比如何？

交付物：用 1-2 句话回答。

2.1.5 混合精度

到目前为止，我们一直以 FP32 精度运行——所有模型参数和激活值都是 `torch.float32` 数据类型。然而，现代 NVIDIA GPU 包含专门用于在低精度下加速矩阵乘法的 GPU 核心（Tensor Core）。例如，NVIDIA B200 的规格表显示其 FP32 最大吞吐量为 80 TFLOPS，而 FP16（半精度浮点数）或 BF16（bfloat16）的最大吞吐量则要高得多，高达 2500 TFLOPS。因此，使用低精度数据类型应当有助于加速训练和推理。

然而，把模型直接转换为低精度格式可能会降低模型的精度。例如，实践中许多梯度值常常小到 FP16 无法表示，于是在用 FP16 精度天真地训练时会变成零。为了解决这个问题，用 FP16 训练时通常会采用损失缩放（loss scaling）——只需把损失乘上一个缩放因子，增大梯度的量级，使其不至于下溢为零。此外，FP16 的动态范围比 FP32 小，可能导致溢出，表现为损失变成 NaN。完整的 bfloat16 训练通常更稳定（因为 BF16 与 FP32 具有相同的动态范围），但与 FP32 相比仍可能影响模型的最终性能。

为了利用低精度数据类型带来的加速，通常会使用*混合精度*（mixed-precision）训练。在 PyTorch 中，它通过 `torch.autocast` 上下文管理器实现。此时，某些运算（例如矩阵乘法）会以低精度数据类型执行，而其他需要 FP32 完整动态范围的运算（例如累加和归约）则保持原样。例如，下面的代码会在前向传播期间自动识别哪些运算应以低精度执行，并把它们转换为指定的数据类型：

```
model : torch.nn.Module = ... # e.g. your Transformer model
dtype : torch.dtype = ... # e.g. torch.bfloat16
x : torch.Tensor = ... # input data

with torch.autocast(device_type="cuda", dtype=dtype):
    y = model(x)
```

如上所述，即使被累加的张量本身已经降低精度，通常也最好让累加保持在高精度下进行。下面的练习有助于你建立对此的直觉。

问题（`mixed_precision_accumulation`）：混合精度累加（1 分）

运行下面的代码，并对结果的精度加以评述。

```
s = torch.tensor(0,dtype=torch.float32)
for i in range(1000):
    s += torch.tensor(0.01,dtype=torch.float32)
print(s)

s = torch.tensor(0,dtype=torch.float16)
for i in range(1000):
```

```
    s += torch.tensor(0.01,dtype=torch.float16)
print(s)

s = torch.tensor(0,dtype=torch.float32)
for i in range(1000):
    s += torch.tensor(0.01,dtype=torch.float16)
print(s)

s = torch.tensor(0,dtype=torch.float32)
for i in range(1000):
    x = torch.tensor(0.01,dtype=torch.float16)
    s += x.type(torch.float32)
print(s)
```

交付物：用 2-3 句话回答。

下面我们先把混合精度应用到一个玩具模型上以建立直觉，然后再应用到我们的基准测试脚本中。

问题（benchmarking_mixed_precision ）：混合精度基准测试（2 分）

(a) 考虑下面的模型：

```
class ToyModel(nn.Module):
    def __init__(self, in_features: int, out_features: int):
        super().__init__()
        self.fc1 = nn.Linear(in_features, 10, bias=False)
        self.ln = nn.LayerNorm(10)
        self.fc2 = nn.Linear(10, out_features, bias=False)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.ln(x)
        x = self.fc2(x)
        return x
```

假设我们在 GPU 上训练该模型，且模型参数原本是 FP32。我们想使用 FP16 的自动转型（autocasting）混合精度。下列各项的数据类型分别是什么：

- autocast 上下文内的模型参数？
- 第一个前馈层（ToyModel.fc1 ）的输出？
- 层归一化（ToyModel.ln ）的输出？
- 模型预测的 logit？
- 损失？
- 模型的梯度？

交付物：上面列出的每个组件的数据类型。

(b) 你应该已经看到，FP16 混合精度 autocast 对层归一化层的处理与前馈层不同。层归一化的哪些部分对混合精度敏感？如果用 BF16 代替 FP16，我们还需要区别对待层归一化吗？为什么？

交付物：用 2-3 句话回答。

(c) 修改你的基准测试脚本，使其可以选择使用 BF16 混合精度运行模型。对第 2.1.2 节描述的每种语言模型规模，分别在使用和不使用混合精度的情况下为前向和反向传播计时。比较全精度与混合精度的结果，并评述模型规模变化时出现的任何趋势。你可能会发现 `nullcontext` 这个空操作上下文管理器很有用。

交付物：用 2-3 句话给出你的计时结果和评述。

2.1.6 内存性能分析

到目前为止，我们关注的是计算性能。现在我们把注意力转向*内存*——语言模型训练和推理中的另一大资源。PyTorch 还自带一个强大的内存分析器，可以跟踪随时间变化的内存分配情况。

要使用内存分析器，可以这样修改你的基准测试脚本：

```
... # warm-up phase in your benchmarking script

# Start recording memory history.
torch.cuda.memory._record_memory_history(max_entries=1000000)

... # what you want to profile in your benchmarking script

# Save a pickle file to be loaded by PyTorch's online tool.
torch.cuda.memory._dump_snapshot("memory_snapshot.pickle")

# Stop recording history.
torch.cuda.memory._record_memory_history(enabled=None)
```

这会输出一个 `memory_snapshot.pickle` 文件，你可以把它加载到在线工具 pytorch.org/memory_viz 中。该工具让你查看总体内存使用时间线，以及每一次单独的内存分配，包括其大小和指向分配来源代码的栈回溯。要使用该工具，你应在网页浏览器中打开上面的链接，然后把 pickle 文件拖放到页面上。

现在你将使用 PyTorch 分析器来分析自己模型的内存使用情况。

问题（`memory_profiling`）：内存性能分析（4 分）

对表 1 中的 xl 模型，在上下文长度为 128 和 2048 的情况下，分析其完整训练步骤（前向传播、反向传播和优化器步骤）的内存使用。

(a) 在你的分析脚本中添加一个选项，让模型通过内存分析器运行。复用你之前的一些基础设施可能有助于此（例如启用混合精度、加载特定模型规模等）。然后运行脚本，得到 xl 模型在仅

推理（只做前向传播）或完整训练步骤时的内存分析结果。你的内存时间线是什么样子？能否根据看到的峰值判断当前正在运行哪个阶段？

交付物：来自 `memory_viz` 工具的 xl 模型“Active memory timeline”的两张图：一张对应前向传播，另一张对应完整训练步骤（前向和反向传播，然后是优化器步骤），并用 2-3 句话加以说明。

(b) 做前向传播时，每个上下文长度的峰值内存使用是多少？做完整训练步骤时呢？

交付物：一张表格，每个上下文长度给出两个数字。

(c) 求出 xl 模型在使用混合精度时的峰值内存使用，前向传播和完整训练步骤都要测。混合精度对内存使用有显著影响吗？

交付物：用 2-3 句话回答。

(d) 考虑 xl 模型。根据我们的参考超参数，Transformer 残差流（residual stream）中的一个激活值张量在单精度下的大小是多少？请用 MiB 给出该大小（即把字节数除以 1024^2）。

交付物：用 1-2 句话给出你的推导。

(e) 现在仔细观察 xl 模型做前向传播的内存快照在 `pytorch.org/memory_viz` 上的“Active Memory Timeline”。当你调低“Detail”级别时，工具会隐藏相应级别以下最小的那些分配（例如把“Detail”设为 10% 时，只显示最大的 10% 的分配）。显示出来的最大分配有多大？通过查看栈回溯，你能说出这些分配来自哪里吗？

交付物：用 1-2 句话回答。

(f) Nsight Systems 也有用于内存分析的标志。你可以把它们与前面的 Nsight 标志结合使用，以了解模型生命周期的不同步骤中发生了哪些分配。利用 PyTorch 提供的 NVTX 标签，确定模型中单个 `TransformerBlock` 为反向传播保存了多少内存（这些张量常被称为残差，residuals）。记录贡献最大的 5 个运算，以及它们占总体内存的百分比。

在反向传播期间，所有这些张量都会被释放，但同时会产生新的梯度张量。根据你的分析结果——前向传播期间分配了多少内存，以及反向传播中每经过一个 `TransformerBlock` 内存使用变化多少——计算为一个 `TransformerBlock` 生成的梯度张量占用多少内存。结果与你的预期相符吗？

交付物：Nsight Systems 的截图以及 1-2 段说明。

3 单 GPU 内存

作业的后续部分将探索把张量分片（shard）到多块 GPU 上的技巧，但也有一些技巧即使对单 GPU 训练同样适用。其中最常见的是梯度检查点（gradient checkpointing，也称为激活检查点，activation checkpointing）。

3.1 自动求导残差

回顾一下：要对模型执行反向传播，我们需要保存在前向传播中产生的激活值。对某些运算来说这显然是必要的，但默认情况下，会这样做的运算比你预期的多得多。为反向传播保存的张量被称为「残差（residuals）」，或简称「已保存张量（saved tensors）」。

让我们来了解网络中到底保存了什么。从这个不起眼的 RMSNorm 函数开始（为简单起见使用纯 FP32），我们添加一些钩子，在张量被自动求导保存或读取时打印信息。

```
import torch
from torch import nn

x = torch.randn((4, 512, 2560), requires_grad=True)

class RMSNorm(nn.Module):
    def __init__(
        self,
        hidden_size: int,
        eps: float = 1e-5,
        device=None,
    ):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(hidden_size, device=device))
        self.eps = eps

    def forward(self, x):
        rms = torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)
        x = x * rms
        return self.weight * x

def pack_hook(t):
    shape, dtype, grad_fn = t.shape, t.dtype, t.grad_fn
    print(f"Saving residual: {shape=}, {dtype=}, {grad_fn=}")
    return t

def unpack_hook(t):
    shape, dtype, grad_fn = t.shape, t.dtype, t.grad_fn
    print(f"Loading residual: {shape=}, {dtype=}, {grad_fn=}")
    return t

ln = RMSNorm(x.shape[-1])

with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = ln(x)
    y.sum().backward()
```

输出显示被写出的张量数量多得令人担忧，其中几个还是完整的激活值大小！

```
$ uv run scripts/autograd_experiment.py
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
```

```
Saving residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
    object at 0x7f7dd319b5e0>
Saving residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
    object at 0x7f7dd319b5e0>
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=<MulBackward0
    object at 0x7f7dd319b5e0>
Saving residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None

Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=<MulBackward0
    object at 0x7f7cf14e6740>
Loading residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
    object at 0x7f7cf14e6740>
Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
    object at 0x7f7cf14e6740>
Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
```

3.1.1 算子融合

在这个例子中，很明显所用运算的粒度太细了。我们想要一个单一的算子：接收 RMSNorm 权重和激活值，直接给出输出，并且该算子在反向传播中作为一个整体（unitary）来处理。这正是核函数融合（kernel fusion）的动机之一。由于 RMSNorm 的行为相当规范，我们甚至可以用 `torch.compile` 自动融合它。

```
...
ln = torch.compile(RMSNorm(x.shape[-1]))

with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = ln(x)
    y.sum().backward()
```

新的输出明显好多了：

```
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=None
```

现在我们只需要为反向传播保存一个完整尺寸的激活值张量——即 RMSNorm 函数的输入。还请注意，读取的顺序不再是保存顺序的逆序，而且每个残差也不再带有 `grad_fn` 依赖——PyTorch 把我们的整个 RMSNorm 当作单个函数来对待。

3.2 激活检查点

融合无疑有用，但在节省内存方面它也只能帮我们到这里。例如，我们来把 xl 规模下的单个 TransformerBlock 做一下融合。

```
import torch
from cs336_basics.model import RotaryEmbedding, TransformerBlock

# num_layers for this model is 32
d_model, d_ff, num_heads, context_length = 2560, 10240, 16, 2048
block = TransformerBlock(d_model=d_model, d_ff=d_ff, num_heads=num_heads,
positional_encoder=RotaryEmbedding(dim=d_model // num_heads, context_length=context_length))

# Fuse as much torch.compile will allow
block = torch.compile(block, fullgraph=True)
x = torch.randn((4, context_length, d_model), requires_grad=True)
```

```
...
# Now logs the number of bytes saved
total_size_bytes = 0
def pack_hook(t):
    if isinstance(t, torch.nn.Parameter):  # Skip logging parameters to avoid double counting
        return t
    global total_size_bytes
    shape, dtype, grad_fn = t.shape, t.dtype, t.grad_fn
    total_size_bytes += t.numel() * t.element_size()
    print(f"Saving residual: {shape=}, {dtype=}, {grad_fn=}")
    return t

...

# Run forward pass, saving for backward
with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = block(x)

print(f"Total size of saved tensors in single TransformerBlock: {total_size_bytes / (1024**2):.2f}
     MiB")
```

这个脚本告诉我们为反向传播保存了多少内存：

```
...
Total size of saved tensors in single TransformerBlock: 3651.31 MiB
```

每一层就是 3.6 GiB。如果对所有层都这样做，仅为反向传播而保存的激活值就高达 114 GiB！注意力运算的残差中有相当可观的浪费，我们将在第 4 节修复这个问题；但即使修复之后，内存使用仍会随批量大小、序列长度和嵌入维度线性增长。

3.2.1 重计算

我们不必保留生成的每一个张量，而是可以只定期保存结果的检查点，并重新计算中间的值。PyTorch 提供了一个接口，可以用简单的方式处理这件事。`torch.utils.checkpoint.checkpoint` 接收一个函数以及传给该函数的参数，然后按如下方式改变传入函数的行为：

1. 在前向传播中：
   (a) 保存传给该函数的输入值；
   (b) 抑制前向传播中张量的保存。
2. 在反向传播中：
   (a) 在前面加上一个重计算步骤：用先前保存的输入重新计算前向传播，并为反向传播保存值；
   (b) 运行反向传播，随后所有张量都可以被释放。

在依次通过 4 个 transformer 块的简单情形中，可以看到内存按我们预期的方式累加。

```
...
def four_blocks(x):
    x = block(x)
    x = block(x)
    x = block(x)
    x = block(x)
    return x

with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = four_blocks(x)

print(f"Total size of saved tensors in four TransformerBlocks: {total_size_bytes / (1024**2):.2f}
    MiB")
```

```
Total size of saved tensors in four TransformerBlocks: 14605.25 MiB
```

但我们可以像下面这样采用梯度检查点：

```
from torch.utils.checkpoint import checkpoint
def two_blocks(x):
    x = block(x)
    x = block(x)
    return x

def four_blocks_checkpoint(x):
    # checkpoint throws out all the saved tensors until the backward pass
    # when getting to the checkpointed block in the backward pass,
    # it reruns a forward pass to produce the saved tensors,
    # then completes normal backward pass.
    x = checkpoint(two_blocks, x, use_reentrant=False)
```

```
    x = checkpoint(two_blocks, x, use_reentrant=False)
    return x

with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = four_blocks_checkpoint(x)

print(f"Total size of saved tensors in four TransformerBlocks with checkpointing: {
    total_size_bytes / (1024**2):.2f} MiB")
```

```
Saving residual: shape=torch.Size([0]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 2048, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([0]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 2048, 2560]), dtype=torch.float32,
grad_fn=<torch.autograd.function.CompiledFunctionBackward object at 0x7aa0657a19d0>
Total size of saved tensors in four TransformerBlocks with checkpointing: 160.00 MiB
```

请注意，这并没有消除内存使用，而是把我们的内存使用分解成了两类：为在每个 `checkpoint` 调用的入口点准备重计算而保存的较长期存储（即检查点本身），以及在检查点块的重计算过程中产生的短期内存，用于支持穿过该块的反向传播。由于我们主要关心的是峰值内存使用，我们希望取得平衡：一方面是保存检查点的内存开销，另一方面是具体化（materialize）完整一个块的残差的内存开销。使用的检查点越多，单个块内具体化的内存就越少，但检查点本身需要的内存就越多。

在上面我们的例子中，显然重计算残差的内存占了主导（部分原因是我们的检查点非常小），所以把平衡更多地向检查点内存倾斜会更有利。因此，我们会想减小检查点块的范围。

检查点块更大或更小并不影响重计算的计算开销，但我们还可以用*递归检查点*（recursive checkpointing）以计算换内存，即把 `checkpoint` 调用嵌套在其他 `checkpoint` 调用内部。

问题（`gradient_checkpointing`）：内存最优的梯度检查点（4 分）

考虑一个由 N 个相同块顺序堆叠而成的 Transformer。在不做任何检查点的情况下，全部 N 个块的残差会同时存活，带来 $O(N)$ 的峰值激活内存。我们可以自由地用 `checkpoint` 包裹前向传播的任意子集，包括把 `checkpoint` 调用相互嵌套。

(a) 在忽略计算开销的情况下，什么检查点策略能使峰值激活内存最小？描述你会如何安排 `checkpoint` 调用（给出代码示意即可），并给出你的策略的渐近峰值激活内存和计算量关于 N 的函数。假设单个块保存的残差主导了每个检查点的簿记开销。

交付物：用 3-5 句话描述该策略及其渐近峰值内存，并附一段简短的代码示意。

(b) 考虑上面批量大小为 4、序列长度为 2048 的 xl 模型配置。如果你的时间/计算预算只够做一步重计算（即不能嵌套 `checkpoint` 调用），那么降低峰值内存的最佳检查点策略是什么？对你运行的峰值内存进行性能分析，以验证你的假设。为确保起见，再比较比它小一档和大一档的检查点块大小下的峰值内存。

交付物：用 3-5 句话描述你的推理，并附上你的策略实测的峰值内存。

4 GPU 核函数

4.1 用 FlashAttention-2 优化注意力

4.1.1 PyTorch 注意力基准测试

你的性能分析结果很可能表明，注意力层在内存和计算两方面都存在优化机会。从高层来看，注意力运算由一次矩阵乘法、随后的 softmax、再一次矩阵乘法构成：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\text{mask}\left(\frac{QK^\top}{\sqrt{d_k}}\right)\right)V \tag{1}$$

朴素的注意力实现需要为每个 batch/head 元素保存形状为 `seq_len` × `seq_len` 的注意力分数矩阵，当序列很长时它会变得非常大，导致任何输入或输出较长的任务出现内存溢出（out-of-memory）错误。我们将按照 FlashAttention-2 论文实现一个注意力核函数：它按分块（tile）计算注意力，从而完全避免显式具体化 `seq_len` × `seq_len` 的注意力分数矩阵，使模型能够扩展到长得多的序列长度。

问题（`pytorch_attention`）：PyTorch 注意力基准测试（2 分）

(a) 在不同规模下对你的注意力实现进行基准测试。编写一个脚本，它将：

(i) 把批量大小固定为 8，不使用多头注意力（即去掉 head 维度）。

(ii) 遍历头嵌入维度 d_{model} 取 `[16, 32, 64, 128]` 与序列长度取 `[256, 1024, 4096, 8192, 16384]` 的笛卡尔积。

(iii) 生成相应大小的随机输入 Q、K、V。

(iv) 使用这些输入为 100 次注意力前向传播计时。

(v) 测量反向传播开始前已使用的内存量，并为 100 次反向传播计时。

(vi) 记得预热，并在每次前向/反向传播后调用 `torch.cuda.synchronize()`。

取决于你的 GPU，其中某些配置预计会发生内存溢出。报告这些配置下你得到的计时结果（或内存溢出错误）。在什么规模下会出现内存溢出错误？对你找到的溢出配置中最小的一个，核算注意力的内存使用（可以使用作业 1 中关于 Transformer 内存占用的公式）。为反向传播保存的内存随序列长度如何变化？你会怎么做来消除这部分内存开销？

交付物：一张包含计时结果和内存使用计算的表格，以及 1-2 段说明。

4.2 JIT 编译注意力的基准测试

从 2.0 版本起，PyTorch 还自带一个强大的即时（just-in-time）编译器，它会自动尝试对 PyTorch 函数应用多种优化：入门介绍参见 https://pytorch.org/tutorials/intermediate/torch_compile_tutorial.html。特别是，它会通过动态分析你的计算图，尝试自动生成融合的 Triton 核函数。使用 PyTorch 编译器的接口非常简单。例如，如果想把它应用于模型中的单个层，可以这样写：

```
layer = SomePyTorchModule(...)
compiled_layer = torch.compile(layer)
```

现在，`compiled_layer` 在功能上的表现与 `layer` 完全一样（包括它的前向和反向传播）。我们也可以用 `torch.compile(model)` 编译整个 PyTorch 模型，甚至编译调用了 PyTorch 运算的 Python 函数。

问题（`torch_compile`）：Torch Compile（2 分）

(a) 扩展你的注意力基准测试脚本，加入你的 PyTorch 注意力实现的一个编译版本，并在与上文 `pytorch_attention` 问题相同的配置下，比较编译版本与未编译版本的性能。

交付物：一张表格，比较你的编译版注意力模块与前文 `pytorch_attention` 问题中未编译版本的前向与反向传播耗时。

(b) 接下来，在你的端到端基准测试脚本中编译整个 Transformer 模型。前向传播的性能有何变化？前向传播加反向传播以及优化器步骤合并起来的性能又如何？

交付物：一张表格，比较你的原始（未编译）版本与编译后的 Transformer 模型。

鉴于我们所看到的随序列长度变化的扩展行为，要处理长序列就需要显著的改进。即使使用 `torch.compile`，当前实现在长序列长度下仍然存在非常糟糕的内存访问模式。为此，我们将编写 FlashAttention-2 的 Triton 实现，从而对内存的访问方式以及何时计算什么拥有大得多的控制权。

4.2.1 示例：加权和

为了介绍关于 Triton 你需要了解的内容以及它与 PyTorch 的互操作方式，我们将通过一个“加权和”（weighted sum）运算的示例核函数来讲解。更多上手 Triton 的资源请参阅 Triton 的官方教程。我们注意到，这些教程并未使用新的、便利的块指针（block pointer）抽象，下面我们将详细介绍它。

给定一个输入矩阵 X，我们将其各元素乘以一个按列的权重向量 w，然后对每一行求和，得到 X 与 w 的矩阵–向量乘积。我们先过一遍该运算的前向传播，然后再编写反向传播的 Triton 核函数。

前向传播

我们核函数的前向传播就是下面这个广播内积。

```
def weighted_sum(x, weight):
    # 这里假设 x 是 n 维张量，形状为 [..., D]，weight 是 1 维张量，形状为 [D]
    return (weight * x).sum(axis=-1)
```

在编写 Triton 核函数时，我们让每个程序实例（可能并行运行）计算 x 的一个行分块（tile）的加权和，并把相应的标量输出写入输出张量。在 Triton 中，一个程序实例是运行同一程序的一块线程（block of threads），这些线程块可以在 GPU 上并行运行。我们不把张量本身作为参数传入，而是传入指向其首元素的指针，以及每个张量的步长（stride），步长告诉我们如何沿各个轴移动。

我们可以利用步长来加载与当前实例正在求和的 x 行分块相对应的张量，并用程序 ID 来划分工作（即实例 i 处理 x 的第 i 个行分块）。在这个简单的例子里，Triton 前向传播与 PyTorch 前向传播的主要区别在于需要进行指针运算以及显式的加载/存储。我们将使用块指针抽象（`tl.make_block_ptr` ）来大幅简化指针运算，不过这意味着需要做一些准备工作来构造块指针。

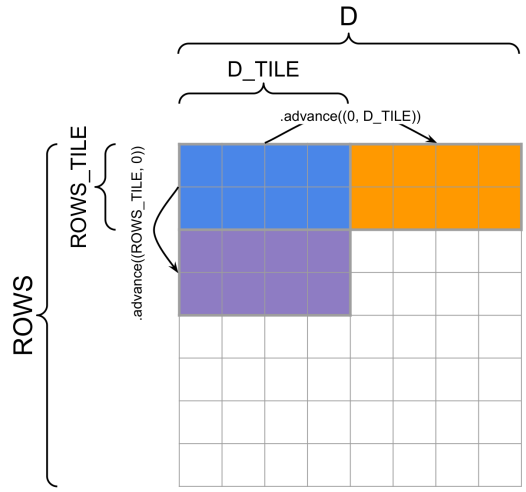


图 2: 加权和核函数示例（第 4.2.1 节）中分块的划分与块指针的推进

图 2 给出了分块以及块指针如何推进的示意图。上面的加权和函数如下所示：

```
import triton
import triton.language as tl

@triton.jit
def weighted_sum_fwd(
    x_ptr, weight_ptr,  # 输入指针
    output_ptr,  # 输出指针
    x_stride_row, x_stride_dim,  # 步长告诉我们如何沿张量每个轴移动一个元素
    weight_stride_dim,  # 很可能是 1
    output_stride_row,  # 很可能是 1
    NUM_ROWS, D,
    ROWS_TILE_SIZE: tl.constexpr, D_TILE_SIZE: tl.constexpr,  # 分块形状必须在编译期已知
):
    # 每个实例将计算 x 的一个行分块的加权和。
    # `tl.program_id` 让我们能够知道自己正运行在哪个线程块中
    row_tile_idx = tl.program_id(0)
    # 块指针让我们可以从一块 N 维内存区域中进行选取，
    #  并移动我们选取的区域。
    # 块指针必须知道：
```

```
# - 指向张量首元素的指针
# - 张量的整体形状，以处理越界访问
# - 每个维度的步长，以正确使用内存布局
# - 起始块的 N 维坐标，即 "offsets"
# - 每次加载/存储的分块形状
# - 各维度在内存中从主要（major）到次要（minor）的顺序
#     （= np.argsort(strides)），用于优化；>=Hopper 架构上的
#     TMA 支持也需要它
x_block_ptr = tl.make_block_ptr(
    x_ptr,
    shape=(NUM_ROWS, D,),
    strides=(x_stride_row, x_stride_dim),
    offsets=(row_tile_idx * ROWS_TILE_SIZE, 0),
    block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE),
    order=(1, 0),
)
weight_block_ptr = tl.make_block_ptr(
    weight_ptr,
    shape=(D,),
    strides=(weight_stride_dim,),
    offsets=(0,),
    block_shape=(D_TILE_SIZE,),
    order=(0,),
)
output_block_ptr = tl.make_block_ptr(
    output_ptr,
    shape=(NUM_ROWS,),
    strides=(output_stride_row,),
    offsets=(row_tile_idx * ROWS_TILE_SIZE,),
    block_shape=(ROWS_TILE_SIZE,),
    order=(0,),
)
# 初始化一个待写入的缓冲区
output = tl.zeros((ROWS_TILE_SIZE,), dtype=tl.float32)
for i in range(tl.cdiv(D, D_TILE_SIZE)):
    # 加载当前的块指针
    # 由于 ROWS_TILE_SIZE 可能无法整除 NUM_ROWS，且 D_TILE_SIZE 可能无法整除 D，
    # 我们需要对两个维度都做边界检查
    row = tl.load(x_block_ptr, boundary_check=(0, 1), padding_option="zero")  # (
ROWS_TILE_SIZE, D_TILE_SIZE)
    weight = tl.load(weight_block_ptr, boundary_check=(0,), padding_option="zero")  # (
D_TILE_SIZE,)
    # 计算该行的加权和。
    output += tl.sum(row * weight[None, :], axis=1)
    # 把指针移动到下一个分块。
    # 这些是 (行，列) 坐标增量
    x_block_ptr = x_block_ptr.advance((0, D_TILE_SIZE))  # 在最后一维上移动 D_TILE_SIZE
    weight_block_ptr = weight_block_ptr.advance((D_TILE_SIZE,))  # 移动 D_TILE_SIZE
# 把输出写到输出块指针（每行一个标量）。
# 由于 ROWS_TILE_SIZE 可能无法整除 NUM_ROWS，我们需要边界检查
```

```
    tl.store(output_block_ptr, output, boundary_check=(0,))
```

接下来，我们把这个核函数包装进一个 PyTorch Autograd 函数，使其能与 PyTorch 互操作（即以张量为输入、输出一个张量，并且之后在反向传播时也能与自动求导引擎协同工作）：

```
class WeightedSumFunc(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, weight):
        # 缓存 x 和 weight 供反向传播使用：届时
        # 我们只会拿到关于输出张量的梯度，而
        # 需要计算关于 x 和 weight 的梯度。
        D, output_dims = x.shape[-1], x.shape[:-1]
        # 把输入张量重塑为 2 维
        input_shape = x.shape
        x = rearrange(x, "... d -> (...) d")
        ctx.save_for_backward(x, weight)
        assert len(weight.shape) == 1 and weight.shape[0] == D, "Dimension mismatch"
        assert x.is_cuda and weight.is_cuda, "Expected CUDA tensors"
        assert x.is_contiguous(), "Our pointer arithmetic will assume contiguous x"
        ctx.D_TILE_SIZE = triton.next_power_of_2(D) // 16  # 大约沿嵌入维度循环 16 次
        ctx.ROWS_TILE_SIZE = 16  # 每个线程一次处理 16 个批次元素
        ctx.input_shape = input_shape
        # 需要初始化一个空的结果张量。注意这些元素不一定是 0!
        y = torch.empty(output_dims, device=x.device)
        # 在 1 维网格中以 n 个实例启动我们的核函数。
        n_rows = y.numel()
        weighted_sum_fwd[(triton.cdiv(n_rows, ctx.ROWS_TILE_SIZE),)](
            x, weight,
            y,
            x.stride(0), x.stride(1),
            weight.stride(0),
            y.stride(0),
            NUM_ROWS=n_rows, D=D,
            ROWS_TILE_SIZE=ctx.ROWS_TILE_SIZE, D_TILE_SIZE=ctx.D_TILE_SIZE,
        )
        return y.view(input_shape[:-1])
```

注意，当我们以 `weighted_sum_fwd[(triton.cdiv(n_rows, ctx.ROWS_TILE_SIZE),)]` 调用 Triton 核函数时，我们通过传入元组 `(triton.cdiv(n_rows, ctx.ROWS_TILE_SIZE),)` 定义了一个所谓的线程块“启动网格”（launch grid）。随后，在核函数内部我们可以用 `tl.program_id(0)` 访问线程块索引。

反向传播

由于我们定义了自己的核函数，我们也需要编写自己的反向函数。

在前向传播中，我们拿到层的输入，需要计算它的输出。而在反向传播中，回想一下，我们会拿到目标函数关于我们输出的梯度，需要计算关于每个输入的梯度。在我们的例子中，我们的运算以

一个矩阵 $\boldsymbol{x} \in \mathbb{R}^{n \times h}$ 和一个权重向量 $\boldsymbol{w} \in \mathbb{R}^{h}$ 作为输入。为简洁起见，把我们的运算记为 $f(\boldsymbol{x}, \boldsymbol{w})$，其值域为 $\mathbb{R}^{n}$。那么，假设给定 $\nabla_{f(\boldsymbol{x},\boldsymbol{w})}\mathcal{L}$，即损失 $\mathcal{L}$ 关于该层输出的梯度，我们可以应用多元链式法则，得到关于 $\boldsymbol{x}$ 和 $\boldsymbol{w}$ 的梯度的如下表达式：

$$(\nabla_{\boldsymbol{x}}\mathcal{L})_{ij} = \sum_{k=1}^{n} \frac{\partial f(\boldsymbol{x}, \boldsymbol{w})_k}{\partial x_{ij}} \left(\nabla_{f(\boldsymbol{x},\boldsymbol{w})}\mathcal{L}\right)_k = w_j \cdot \left(\nabla_{f(\boldsymbol{x},\boldsymbol{w})}\mathcal{L}\right)_i \tag{2}$$

$$(\nabla_{\boldsymbol{w}}\mathcal{L})_{j} = \sum_{i=1}^{n} \frac{\partial f(\boldsymbol{x}, \boldsymbol{w})_i}{\partial w_j} \left(\nabla_{f(\boldsymbol{x},\boldsymbol{w})}\mathcal{L}\right)_i = \sum_{i=1}^{n} x_{ij} \cdot \left(\nabla_{f(\boldsymbol{x},\boldsymbol{w})}\mathcal{L}\right)_i \tag{3}$$

这为计算反向传播给出了一个简单的公式。要得到关于 $\boldsymbol{x}$ 的反向更新，我们应用式 (2)，取 $\boldsymbol{w}$ 与 $\nabla_{f(\boldsymbol{x},\boldsymbol{w})}\mathcal{L}$ 的外积。要计算关于 $\boldsymbol{w}$ 的反向更新（即 $(\nabla_{\boldsymbol{w}}\mathcal{L})_j$），我们必须把输入梯度乘以相应的输出行。

我们的反向传播核函数首先定义所有块指针，然后计算 $\nabla_{\boldsymbol{x}}\mathcal{L}$：

```
@triton.jit
def weighted_sum_backward(
    x_ptr, weight_ptr,  # 输入
    grad_output_ptr,  # 梯度输入
    grad_x_ptr, partial_grad_weight_ptr,  # 梯度输出
    stride_xr, stride_xd,
    stride_wd,
    stride_gr,
    stride_gxr, stride_gxd,
    stride_gwb, stride_gwd,
    NUM_ROWS, D,
    ROWS_TILE_SIZE: tl.constexpr, D_TILE_SIZE: tl.constexpr,
):
    row_tile_idx = tl.program_id(0)
    n_row_tiles = tl.num_programs(0)
    # 输入
    grad_output_block_ptr = tl.make_block_ptr(
        grad_output_ptr,
        shape=(NUM_ROWS,), strides=(stride_gr,),
        offsets=(row_tile_idx * ROWS_TILE_SIZE,),
        block_shape=(ROWS_TILE_SIZE,),
        order=(0,),
    )
    x_block_ptr = tl.make_block_ptr(
        x_ptr,
        shape=(NUM_ROWS, D,), strides=(stride_xr, stride_xd),
        offsets=(row_tile_idx * ROWS_TILE_SIZE, 0),
        block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE),
        order=(1, 0),
    )
    weight_block_ptr = tl.make_block_ptr(
        weight_ptr,
        shape=(D,), strides=(stride_wd,),
```

```
        offsets=(0,), block_shape=(D_TILE_SIZE,),
        order=(0,),
    )
    grad_x_block_ptr = tl.make_block_ptr(
        grad_x_ptr,
        shape=(NUM_ROWS, D,), strides=(stride_gxr, stride_gxd),
        offsets=(row_tile_idx * ROWS_TILE_SIZE, 0),
        block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE),
        order=(1, 0),
    )
    partial_grad_weight_block_ptr = tl.make_block_ptr(
        partial_grad_weight_ptr,
        shape=(n_row_tiles, D,), strides=(stride_gwb, stride_gwd),
        offsets=(row_tile_idx, 0),
        block_shape=(1, D_TILE_SIZE),
        order=(1, 0),
    )
    for i in range(tl.cdiv(D, D_TILE_SIZE)):
        grad_output = tl.load(grad_output_block_ptr, boundary_check=(0,), padding_option="zero")
    # (ROWS_TILE_SIZE,)
        # 用外积计算 grad_x
        weight = tl.load(weight_block_ptr, boundary_check=(0,), padding_option="zero")  # (
    D_TILE_SIZE,)
        grad_x_row = grad_output[:, None] * weight[None, :]
        tl.store(grad_x_block_ptr, grad_x_row, boundary_check=(0, 1))
        # 对 grad_weight 结果尽可能多地进行行归约
        row = tl.load(x_block_ptr, boundary_check=(0, 1), padding_option="zero")  # (
    ROWS_TILE_SIZE, D_TILE_SIZE)
        grad_weight_row = tl.sum(row * grad_output[:, None], axis=0, keep_dims=True)
        tl.store(partial_grad_weight_block_ptr, grad_weight_row, boundary_check=(1,))  # 在维度 0
    上永远不会越界
        # 把指针沿 D 移动到下一个分块
        x_block_ptr = x_block_ptr.advance((0, D_TILE_SIZE))
        weight_block_ptr = weight_block_ptr.advance((D_TILE_SIZE,))
        partial_grad_weight_block_ptr = partial_grad_weight_block_ptr.advance((0, D_TILE_SIZE))
        grad_x_block_ptr = grad_x_block_ptr.advance((0, D_TILE_SIZE))
```

计算 $\nabla_{\boldsymbol{x}}$ 很简单，我们把结果写到输出张量相应的分块上即可。然而，计算 $\nabla_{\boldsymbol{w}}$ 稍微棘手一些。每个核函数实例负责 x 的一个行分块，但我们现在需要沿 x 的行求和。我们不直接在反向传播中做这个求和，而是约定 `partial_grad_weight_ptr` 指向一个 `n_row_tiles` $\times H$ 的矩阵，其中第一个维度只在 x 的一个行分块内归约：我们在写入该张量之前，先在当前行分块内归约。在核函数之外，我们再用 `torch.sum` 对 $\nabla_{\boldsymbol{w}}$ 做归约，把各个行分块的结果加起来。[1]

`autograd.Function` 的最后一部分就相对简单了：

```
class WeightedSumFunc(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, weight):
```

[1] 当然，我们也可以为此专门编写自己的核函数。

```
        # ...（定义见前文）
    @staticmethod
    def backward(ctx, grad_out):
        x, weight = ctx.saved_tensors
        ROWS_TILE_SIZE, D_TILE_SIZE = ctx.ROWS_TILE_SIZE, ctx.D_TILE_SIZE  # 它们不必相同
        n_rows, D = x.shape
        # 我们的策略是：每个线程块先写入一个部分和缓冲区，
        # 然后我们对该缓冲区做归约，得到最终梯度。
        partial_grad_weight = torch.empty((triton.cdiv(n_rows, ROWS_TILE_SIZE), D), device=x.
    device, dtype=x.dtype)
        grad_x = torch.empty_like(x)
        weighted_sum_backward[(triton.cdiv(n_rows, ROWS_TILE_SIZE),)](
            x, weight,
            grad_out,
            grad_x, partial_grad_weight,
            x.stride(0), x.stride(1),
            weight.stride(0),
            grad_out.stride(0),
            grad_x.stride(0), grad_x.stride(1),
            partial_grad_weight.stride(0), partial_grad_weight.stride(1),
            NUM_ROWS=n_rows, D=D,
            ROWS_TILE_SIZE=ROWS_TILE_SIZE, D_TILE_SIZE=D_TILE_SIZE,
        )
        grad_weight = partial_grad_weight.sum(axis=0)
        return grad_x, grad_weight
```

最后，我们现在可以得到一个用起来很像 `torch.nn.functional` 中那些函数的函数：

```
f_weightedsum = WeightedSumFunc.apply
```

现在，对两个 PyTorch 张量 x 和 w 调用 `f_weightedsum` 会得到如下形式的张量：

```
tensor([ 90.8563, -93.6815, -80.8884,  ..., 103.4840, -21.4634, -24.0192],
       device='cuda:0', grad_fn=<WeightedSumFuncBackward>)
```

注意张量上附带的 `grad_fn` ——它表明当这个张量出现在计算图中时，PyTorch 知道在反向传播时该调用什么。至此，我们完成了加权和运算的 Triton 实现。

4.2.2 FlashAttention-2 前向传播

你将用一个显著改进的、遵循 FlashAttention-2 [T. Dao, 2023] 的 Triton 实现来替换你的 PyTorch 注意力实现。FlashAttention-2 采用一些技巧按分块（tile）计算前向传播，从而获得高效的内存访问模式，并避免在全局内存中具体化完整的注意力矩阵。

在进入本节之前，我们强烈建议至少读一遍原始的 FlashAttention 论文 [T. Dao et al., 2022]，它会让你直观理解 FlashAttention 实现高效注意力的核心技术：以在线（online）方式跨分块计算 softmax（该技巧由 [M. Milakov et al., 2018] 提出）。我们还推荐阅读 H. He [4]，以便更直观地了解 GPU 实际上是如何执行 PyTorch 代码的。

理解朴素注意力的低效之处

回想一下，注意力的前向传播（暂时忽略掩码）可以写为：

$$\boldsymbol{S} = \boldsymbol{Q}\boldsymbol{K}^\top/\sqrt{d} \tag{4}$$

$$P_{ij} = \mathrm{softmax}_j(\boldsymbol{S})_{ij} \tag{5}$$

$$\boldsymbol{O} = \boldsymbol{P}\boldsymbol{V} \tag{6}$$

标准反向传播为

$$\boldsymbol{dV} = \boldsymbol{P}^\top \boldsymbol{dO} \tag{7}$$

$$\boldsymbol{dP} = \boldsymbol{dO}\boldsymbol{V}^\top \tag{8}$$

$$\boldsymbol{dS}_i = \mathrm{dsoftmax}(\boldsymbol{dP}_i) = \left(\mathrm{diag}(\boldsymbol{P}_i) - \boldsymbol{P}_i\boldsymbol{P}_i^\top\right)\boldsymbol{dP}_i \tag{9}$$

$$\boldsymbol{dQ} = \boldsymbol{dS}\boldsymbol{K}/\sqrt{d} \tag{10}$$

$$\boldsymbol{dK} = \boldsymbol{dS}^\top\boldsymbol{Q}/\sqrt{d} \tag{11}$$

正如你所见，反向传播依赖前向传播中一些非常大的激活值。例如，式 (7) 中计算 $\boldsymbol{dV}$ 需要 $\boldsymbol{P}$，即形状为 `(batch_size, n_heads, seq_len, seq_len)` 的注意力分数——这个激活矩阵的大小随序列长度平方级增长，这正好解释了我们之前在大序列长度下对注意力做基准测试时遇到的内存问题。在朴素注意力的前向与反向传播过程中，为在片上 SRAM 与 GPU HBM 之间搬运 $\boldsymbol{P}$ 及其他大型激活值，我们要付出可观的内存 IO 代价。标准实现中存在多次这样的传输：例如，标准的反向传播实现会在式 (7) 和式 (9) 的计算中都从 HBM 读取 $\boldsymbol{P}$。

FlashAttention 的主要目标是避免把注意力矩阵在 HBM 中读入写出，以降低 IO 和峰值内存开销。我们通过三种技术来做到这一点：分块（tiling）、重计算（recomputation）和算子融合（operator fusion）。

分块

为了避免在 HBM 中读写注意力矩阵，我们在无法一次性访问整个输入的情况下计算 softmax 归约。具体而言，我们重构注意力计算，把输入切分成若干分块，并对输入分块进行多趟处理，从而增量式地执行 softmax 归约。

重计算

我们避免在 HBM 中存储形状为 `(batch_size, n_heads, seq_len, seq_len)` 的大型中间注意力矩阵。作为替代，我们在 HBM 中保存一些“激活检查点”，然后在反向传播期间重计算前向传播的一部分，以得到计算梯度所需的其他激活值。FlashAttention-2 还会保存注意力分数的 logsumexp 值 L，用于简化反向传播的计算。L 的表达式为：

$$L_i = \log\left(\sum_j \exp(S_{ij})\right) \tag{12}$$

在我们的最终核函数中，我们会以在线的方式计算它，但最终结果应当相同。结合分块与重计算，我们的内存 IO 和峰值内存占用不再依赖于 sequence_length^2，因此我们可以使用更大的序列长度。

算子融合

最后，我们把所有运算放进单个核函数中执行，从而避免对注意力矩阵和其他中间激活值的反复内存 IO——这称为算子融合（operator fusion）或核函数融合（kernel fusion）。我们将为前向传播编写一个单独的 Triton 核函数，在 HBM 与 SRAM 之间仅进行有限数据传输的情况下完成注意力涉及的全部运算。算子融合在一定程度上得益于重计算：这样我们就可以省去为保存每个中间激活值而付出的内存 IO。

想要对这些技巧有更多直观理解，可以参阅 FlashAttention 论文 [T. Dao et al., 2022; T. Dao, 2023]。

带重计算的反向传播

利用 L，我们可以进行恰当的重计算并高效地计算反向传播。在开始反向传播之前，我们先在全局内存中预计算 $D = \text{rowsum}(\boldsymbol{O} \circ \boldsymbol{dO})$（其中 $\circ$ 表示逐元素相乘），它等于 $\text{rowsum}(\boldsymbol{P} \circ \boldsymbol{dP})$，因为 $\boldsymbol{PdP}^\top = \boldsymbol{P}(\boldsymbol{dOV}^\top)^\top = (\boldsymbol{PV})\boldsymbol{dO}^\top = \boldsymbol{OdO}^\top$（且对任意矩阵 $\boldsymbol{A}$、$\boldsymbol{B}$ 有 $\text{rowsum}(\boldsymbol{A} \circ \boldsymbol{B}) = \text{diag}(\boldsymbol{AB}^\top)$）。有了 L 和 D 这两个向量，反向传播的计算就不再需要 softmax。完整的反向传播计算现在是：

$$\boldsymbol{S} = \boldsymbol{QK}^\top / \sqrt{d} \tag{13}$$

$$P_{ij} = \exp(S_{ij} - L_i) \tag{14}$$

$$\boldsymbol{dV} = \boldsymbol{P}^\top \boldsymbol{dO} \tag{15}$$

$$\boldsymbol{dP} = \boldsymbol{dOV}^\top \tag{16}$$

$$dS_{ij} = P_{ij}\,(dP_{ij} - D_i) \tag{17}$$

$$\boldsymbol{dQ} = \boldsymbol{dSK} / \sqrt{d} \tag{18}$$

$$\boldsymbol{dK} = \boldsymbol{dS}^\top \boldsymbol{Q} / \sqrt{d} \tag{19}$$

可以看到，这一串运算不再要求我们在前向传播期间把注意力分数 $\boldsymbol{P}$ 存入 HBM——我们在式 (13) 和式 (14) 中从激活值 $\boldsymbol{Q}$、$\boldsymbol{K}$ 和 L 重计算它们。

FlashAttention 前向传播的细节

现在我们对 FlashAttention-2 使用的技巧有了高层认识，接下来深入了解你将要实现的 FA2 前向传播核函数的细节。为了避免在 HBM 中读写注意力矩阵，我们希望使用分块，也就是说，独立于其他分块地计算输出的每一个分块。这要求我们能够对 $\boldsymbol{P}$ 分块计算，理想情况下在两个维度（query 和 key）上都分块。

然而，当我们对 $\boldsymbol{S}$ 应用 softmax 时，需要对 $\boldsymbol{S}$ 的整行做归约来计算 softmax 的分母，这意味着我们无法直接分块计算 $\boldsymbol{P}$。FlashAttention-2 使用在线 softmax（online softmax）解决了这个问题。在下文中，我们用下标 i 表示当前 query 分块，用上标 (j) 表示当前 key 分块。沿 query 维度的分块大小为 B_q，沿 key 维度的分块大小为 B_k。我们不会沿隐藏维度 d 分块。

我们还维护一些按行运算的运行值 $\boldsymbol{m}_i^{(j)} \in \mathbb{R}^{B_q}$ 和 $\boldsymbol{l}_i^{(j)} \in \mathbb{R}^{B_q}$。按行的 $\boldsymbol{m}_i^{(j)}$ 值是一个运行最大值，记录它使我们能够以数值稳定的方式计算 softmax（回忆一下作业 1 中 softmax 实现里的这个技巧）。当 j 增大时（即遇到 $\boldsymbol{S}$ 的新行分块），我们会更新 $\boldsymbol{m}_i^{(j)}$。利用这个运行最大值，我们可以把未归一化的 softmax 值（分子）计算为 $\tilde{\boldsymbol{P}}_i^{(j)} = \exp(\boldsymbol{S}_{ij} - \boldsymbol{m}_i^{(j)})$。$\boldsymbol{l}_i^{(j)}$ 是 softmax 分母的运行代理值，它将用未归一化的 softmax 值更新为 $\boldsymbol{l}_i^{(j)} = \exp(\boldsymbol{m}_i^{(j-1)} - \boldsymbol{m}_i^{(j)}) \cdot \boldsymbol{l}_i^{(j-1)} + \mathrm{rowsum}(\tilde{\boldsymbol{P}}_i^{(j)})$。最终写出输出时，我们需要用 $\boldsymbol{l}_i^{(T_k)}$（处理完全部 key 分块后 $\boldsymbol{l}_i^{(j)}$ 的最终值）完成归一化。算法 1 展示了 GPU 上的前向传播。

算法 1：FlashAttention-2 的前向传播

1. **需要：** $\boldsymbol{Q} \in \mathbb{R}^{N_q \times d}$，$\boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{N_k \times d}$，分块大小 B_q、B_k
2. 把 $\boldsymbol{Q}$ 划分为 $T_q = \lceil \frac{N_q}{B_q} \rceil$ 个大小为 $B_q \times d$ 的分块 $\boldsymbol{Q}_1, \ldots, \boldsymbol{Q}_{T_q}$
3. 把 $\boldsymbol{K}, \boldsymbol{V}$ 划分为 $T_k = \lceil \frac{N_k}{B_k} \rceil$ 个大小为 $B_k \times d$ 的分块 $\boldsymbol{K}^{(1)}, \ldots, \boldsymbol{K}^{(T_k)}$ 与 $\boldsymbol{V}^{(1)}, \ldots, \boldsymbol{V}^{(T_k)}$
4. **for** $i = 1, \ldots, T_q$ **do**
5. 　从全局内存加载 $\boldsymbol{Q}_i$
6. 　初始化 $\boldsymbol{O}_i^{(0)} = \boldsymbol{0} \in \mathbb{R}^{B_q \times d}$，$\boldsymbol{l}_i^{(0)} = 0 \in \mathbb{R}^{B_q}$，$\boldsymbol{m}_i^{(0)} = -\infty \in \mathbb{R}^{B_q}$
7. 　**for** $j = 1, \ldots, T_k$ **do**
8. 　　从全局内存加载 $\boldsymbol{K}^{(j)}, \boldsymbol{V}^{(j)}$
9. 　　计算 pre-softmax 注意力分数分块 $\boldsymbol{S}_i^{(j)} = \frac{\boldsymbol{Q}_i (\boldsymbol{K}^{(j)})^\top}{\sqrt{d}} \in \mathbb{R}^{B_q \times B_k}$
10. 　　计算 $\boldsymbol{m}_i^{(j)} = \max\left(\boldsymbol{m}_i^{(j-1)}, \mathrm{rowmax}(\boldsymbol{S}_i^{(j)})\right) \in \mathbb{R}^{B_q}$
11. 　　计算 $\tilde{\boldsymbol{P}}_i^{(j)} = \exp\left(\boldsymbol{S}_i^{(j)} - \boldsymbol{m}_i^{(j)}\right) \in \mathbb{R}^{B_q \times B_k}$
12. 　　计算 $\boldsymbol{l}_i^{(j)} = \exp\left(\boldsymbol{m}_i^{(j-1)} - \boldsymbol{m}_i^{(j)}\right) \boldsymbol{l}_i^{(j-1)} + \mathrm{rowsum}\left(\tilde{\boldsymbol{P}}_i^{(j)}\right) \in \mathbb{R}^{B_q}$
13. 　　计算 $\boldsymbol{O}_i^{(j)} = \mathrm{diag}\left(\exp\left(\boldsymbol{m}_i^{(j-1)} - \boldsymbol{m}_i^{(j)}\right)\right) \boldsymbol{O}_i^{(j-1)} + \tilde{\boldsymbol{P}}_i^{(j)} \boldsymbol{V}^{(j)}$
14. 　**end for**
15. 　计算 $\boldsymbol{O}_i = \mathrm{diag}\left(\boldsymbol{l}_i^{(T_k)}\right)^{-1} \boldsymbol{O}_i^{(T_k)}$
16. 　计算 $L_i = \boldsymbol{m}_i^{(T_k)} + \log\left(\boldsymbol{l}_i^{(T_k)}\right)$
17. 　把 $\boldsymbol{O}_i$ 作为 $\boldsymbol{O}$ 的第 i 个分块写入全局内存。
18. 　把 L_i 作为 L 的第 i 个分块写入全局内存。
19. **end for**
20. 返回输出 $\boldsymbol{O}$ 与 logsumexp L。

在开始用 Triton 实现前向传播之前，我们在这里汇总一些编写 Triton 核函数的通用技巧与窍门。

Triton 技巧与窍门

- 你可以在 Triton 中使用 `tl.device_print` 打印语句来调试：https://triton-lang.org/main/python-api/generated/triton.language.device_print.html。还有一个设置 `TRITON_INTERPRET=1` ，可以在 CPU 上运行 Triton 解释器，不过我们发现它 bug 不少。
- 定义块指针时，务必确保它们的偏移量（offsets）正确，并且块偏移乘上了相应的分块大小。
- 线程块的启动网格通过如下方式设置：

```
kernel_fn[(launch_grid_d1, launch_grid_d2, ...)](...arguments...)
```

  即在 `torch.autograd.Function` 子类的方法中这样调用，正如我们在加权和示例中看到的那样。
- 用 `tl.dot` 执行矩阵乘法。
- 要推进块指针，使用 `*_block_ptr = *_block_ptr.advance(...)` 。

问题（`flash_forward`）：FlashAttention-2 前向传播（15 分）

(a) 编写一个纯 PyTorch（不用 Triton）的 `autograd.Function` 来实现 FlashAttention-2 前向传播。它会比常规的 PyTorch 实现慢很多，但能帮助你调试 Triton 核函数。

你的实现应接受输入 $\boldsymbol{Q}$、$\boldsymbol{K}$、$\boldsymbol{V}$ 以及一个标志 `is_causal` ，并产生输出 $\boldsymbol{O}$ 和 logsumexp 值 L。本题中你可以忽略 `is_causal` 标志。`autograd.Function` 的 forward 随后应为反向传播保存 L、$\boldsymbol{Q}$、$\boldsymbol{K}$、$\boldsymbol{V}$、$\boldsymbol{O}$，并返回 $\boldsymbol{O}$。记住，`autograd.Function` 的 forward 方法的实现总是以上下文（context）作为第一个参数。任何 `autograd.Function` 类都需要实现一个 backward 方法，但现在你可以让它直接抛出 `NotImplementedError` 。如果你需要对比的对象，可以在 PyTorch 中实现式 (4) 到式 (6) 以及式 (12)，并比较你的输出。

接口为 `def forward(ctx, Q, K, V, is_causal=False)` 。分块大小自行确定，但要确保至少为 16×16。我们测试你的代码时使用的维度总是 2 的幂且至少为 16，所以你不必担心越界访问。

交付物：一个实现了 FlashAttention-2 前向传播的 `torch.autograd.Function` 子类。要测试你的代码，实现 `[adapters.get_flashattention_autograd_function_pytorch]` 。然后运行 `uv run pytest -k test_flash_forward_pass_pytorch` ，确保你的实现通过测试。

(b) 按照算法 1，为 FlashAttention-2 的前向传播编写一个 Triton 核函数。然后编写另一个 `torch.autograd.Function` 子类，让它在 forward 中调用这个（融合的）核函数，而不是在 PyTorch 中计算结果。几个针对本题的提示：

- 调试时，建议把你执行的每个 Triton 运算的结果与你在第 (a) 小问中编写的分块 PyTorch 实现进行比较。
- 你的启动网格应设为 $(T_q, \text{batch_size})$，也就是说，每个 Triton 程序实例只加载单个 batch 索引的元素，并且只对 $\boldsymbol{Q}$、$\boldsymbol{O}$ 和 L 的单个 query 分块进行读/写。

- 核函数应该只有一个循环，迭代 key 分块 $1 \leq j \leq T_k$。
- 在循环末尾推进块指针。
- 使用下面的函数声明（借助我们给你的块指针，你应该能推断出其余指针的设置）：

```
@triton.jit
def flash_fwd_kernel(
    Q_ptr, K_ptr, V_ptr,
    O_ptr, L_ptr,
    stride_qb, stride_qq, stride_qd,
    stride_kb, stride_kk, stride_kd,
    stride_vb, stride_vk, stride_vd,
    stride_ob, stride_oq, stride_od,
    stride_lb, stride_lq,
    N_QUERIES, N_KEYS,
    scale,
    D: tl.constexpr,
    Q_TILE_SIZE: tl.constexpr,
    K_TILE_SIZE: tl.constexpr,
):
    # 程序索引
    query_tile_index = tl.program_id(0)
    batch_index = tl.program_id(1)
    # 用对应的批次索引乘以每个张量的批次步长，来偏移各个指针
    Q_block_ptr = tl.make_block_ptr(
        Q_ptr + batch_index * stride_qb,
        shape=(N_QUERIES, D),
        strides=(stride_qq, stride_qd),
        offsets=(query_tile_index * Q_TILE_SIZE, 0),
        block_shape=(Q_TILE_SIZE, D),
        order=(1, 0),
    )
    ...
```

其中 `scale` 为 $1/\sqrt{d}$，`Q_TILE_SIZE` 和 `K_TILE_SIZE` 分别是 B_q 和 B_k。你稍后可以调整它们。

下面这些附加准则可以帮助你避免精度问题：

- 片上缓冲区（$\boldsymbol{O}_i$、$\boldsymbol{l}$、$\boldsymbol{m}$）的 dtype 应为 `tl.float32`。如果你在向一个输出缓冲区累加，请使用 acc 参数（`acc = tl.dot(..., acc=acc)`）。
- 在相乘之前，把 $\tilde{\boldsymbol{P}}_i^{(j)}$ 转换为 $\boldsymbol{V}^{(j)}$ 的 dtype；在把 $\boldsymbol{O}_i$ 写入全局内存之前，把它转换为合适的 dtype。类型转换用 `tensor.to` 完成。你可以用 `tensor.dtype` 获取张量的 dtype，用 `*_block_ptr.type.element_ty` 获取块指针/指针的 dtype。

交付物：一个 `torch.autograd.Function` 子类，它使用你的 Triton 核函数实现 FlashAttention-2 前向传播。实现 `[adapters.get_flash_autograd_function_triton]`。然后运行 `uv run pytest -k test_flash_forward_pass_triton`，确保你的实现通过测试。

(c) 在你的 `autograd.Function` 实现中，把一个标志作为最后一个参数加入，用于因果掩码（causal masking）。它应是一个布尔标志，设为 `True` 时启用用于因果掩码的索引比较。你的 Triton 核函数应有对应的额外参数 `is_causal: tl.constexpr`（这是必需的类型标注）。在 Triton 中，为 query 和 key 构造合适的索引向量，并比较它们以形成大小为 $B_q \times B_k$ 的方形掩码。对于被掩码掉的元素，给注意力分数矩阵 $\boldsymbol{S}_i^{(j)}$ 的相应元素加上常数值 $-1e6$。务必用 `ctx.is_causal = is_causal` 把掩码标志保存下来供反向传播使用。

交付物：为你的 `torch.autograd.Function` 子类添加一个额外的标志，使用你的 Triton 核函数实现带因果掩码的 FlashAttention-2 前向传播。确保该标志是可选的且默认为 `False`，这样之前的测试仍能通过。

用重计算实现反向传播

注意，与式 (7) 到式 (11) 中的标准反向传播不同，我们可以利用重计算来避免式 (13) 到式 (19) 所示反向传播中的 softmax 运算。这意味着我们可以用一个简单的（trivial）核函数来计算反向传播，不需要任何在线技巧。因此在这一部分，你可以通过对一个常规 PyTorch 函数（非 Triton）调用 `torch.compile` 来实现反向传播。

问题（`flash_backward`）：FlashAttention-2 反向传播（5 分）

使用 PyTorch（非 Triton）和 `torch.compile` 为你的 FlashAttention-2 `autograd.Function` 实现反向传播。你的实现应接受 $\boldsymbol{Q}$、$\boldsymbol{K}$、$\boldsymbol{V}$、$\boldsymbol{O}$、$\boldsymbol{dO}$ 和 L 张量作为输入，并返回 $\boldsymbol{dQ}$、$\boldsymbol{dK}$ 和 $\boldsymbol{dV}$。记得计算并使用 D 向量。你可以按照式 (13) 到式 (19) 的计算来实现。

交付物：要测试你的实现，运行 `uv run pytest -k test_flash_backward`。

现在，让我们把你的（部分）Triton 版 FlashAttention-2 实现与你的常规 Attention 的 PyTorch 实现进行性能比较。

问题（`flash_benchmarking`）：FlashAttention-2 基准测试（5 分）

(a) 使用 `triton.testing.do_bench` 编写一个基准测试脚本，把你（部分）Triton 版 FlashAttention-2 的前向和反向传播性能与常规 PyTorch 实现（即不使用 FlashAttention）进行比较。

具体而言，你要报告一张表格，其中包含你的 Triton 实现与 PyTorch 实现各自的前向、反向以及端到端前向–反向传播的延迟。在开始基准测试之前，随机生成一切必要的输入，并在单块 B200 上运行基准测试。始终使用批量大小 1 和因果掩码。遍历如下设定的笛卡尔积：序列长度取从 128 到 65536 的各种 2 的幂，嵌入维度大小取从 16 到 128 的各种 2 的幂，精度取 `torch.bfloat16` 和 `torch.float32`。你可能需要根据输入大小调整分块大小。

交付物：一张结果表格，按上述设定比较你的 FlashAttention-2 实现与 PyTorch 实现，报告前向、反向和端到端延迟。

4.2.3 可选：Triton 反向传播

如果你有兴趣获得更多 Triton 练习，和/或想提交一个快速的成绩榜（leaderboard）结果，我们在下面提供可以用 Triton 实现的分块 FlashAttention-2 反向传播。算法 2 展示了在 Triton 中应当如何实现 FlashAttention-2 的反向传播。这里的一个关键技巧是把 $\boldsymbol{P}$ 计算两次：一次用于 $\boldsymbol{dQ}$，再一次用于 $\boldsymbol{dK}$ 和 $\boldsymbol{dV}$。这让我们无需跨线程块同步，从而可以避免缓慢的原子操作（atomics）。

算法 2：分块的 FlashAttention-2 反向传播

需要： $\boldsymbol{Q}, \boldsymbol{O}, \boldsymbol{dO} \in \mathbb{R}^{N_q \times d}$，$\boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{N_k \times d}$，$L \in \mathbb{R}^{N_q}$，分块大小 B_q、B_k

计算 $D = \mathrm{rowsum}(\boldsymbol{O} \circ \boldsymbol{dO}) \in \mathbb{R}^{N_q}$

把 $\boldsymbol{Q}, \boldsymbol{O}, \boldsymbol{dO}$ 划分为 $T_q = \lceil \frac{N_q}{B_q} \rceil$ 个分块 $\boldsymbol{Q}_1, \ldots, \boldsymbol{Q}_{T_q}$、$\boldsymbol{O}_1, \ldots, \boldsymbol{O}_{T_q}$、$\boldsymbol{dO}_1, \ldots, \boldsymbol{dO}_{T_q}$，每个大小为 $B_q \times d$

把 $\boldsymbol{K}, \boldsymbol{V}$ 划分为 $T_k = \lceil \frac{N_k}{B_k} \rceil$ 个分块 $\boldsymbol{K}^{(1)}, \ldots, \boldsymbol{K}^{(T_k)}$ 与 $\boldsymbol{V}^{(1)}, \ldots, \boldsymbol{V}^{(T_k)}$，每个大小为 $B_k \times d$

把 L, D 划分为 T_q 个分块 $L_1, \ldots, L_{T_q}$ 与 $D_1, \ldots, D_{T_q}$，每个大小为 B_q

for $j = 1, \ldots, T_k$ **do**

  从全局内存加载 $\boldsymbol{K}^{(j)}, \boldsymbol{V}^{(j)}$

  初始化 $\boldsymbol{dK}_0^{(j)} = \boldsymbol{dV}_0^{(j)} = \boldsymbol{0} \in \mathbb{R}^{B_k \times d}$

  for $i = 1, \ldots, T_q$ **do**

    从全局内存加载 $\boldsymbol{Q}_i, \boldsymbol{dO}_i$

    计算注意力分数分块 $\boldsymbol{S}_i^{(j)} = \frac{\boldsymbol{Q}_i (\boldsymbol{K}^{(j)})^\top}{\sqrt{d}} \in \mathbb{R}^{B_q \times B_k}$

    计算注意力概率 $\boldsymbol{P}_i^{(j)} = \exp\left(\boldsymbol{S}_i^{(j)} - L_i\right) \in \mathbb{R}^{B_q \times B_k}$

    计算 $\boldsymbol{dV}_i^{(j)} = \boldsymbol{dV}_{i-1}^{(j)} + \left(\boldsymbol{P}_i^{(j)}\right)^\top \boldsymbol{dO}_i \in \mathbb{R}^{B_k \times d}$

    计算 $\boldsymbol{dP}_i^{(j)} = \boldsymbol{dO}_i \left(\boldsymbol{V}^{(j)}\right)^\top \in \mathbb{R}^{B_q \times B_k}$

    计算 $\boldsymbol{dS}_i^{(j)} = \boldsymbol{P}_i^{(j)} \circ \left(\boldsymbol{dP}_i^{(j)} - D_i\right) \in \mathbb{R}^{B_q \times B_k}$

    计算 $\boldsymbol{dK}_i^{(j)} = \boldsymbol{dK}_{i-1}^{(j)} + \left(\boldsymbol{dS}_i^{(j)}\right)^\top \boldsymbol{Q}_i / \sqrt{d} \in \mathbb{R}^{B_k \times d}$

  end for

  把 $\boldsymbol{dK}_{T_q}^{(j)}$ 和 $\boldsymbol{dV}_{T_q}^{(j)}$ 写入全局内存，作为 $\boldsymbol{dK}$ 和 $\boldsymbol{dV}$ 的第 j 个分块。

end for

for $i = 1, \ldots, T_q$ **do**

  从全局内存加载 $\boldsymbol{Q}_i, \boldsymbol{dO}_i$

  初始化 $\boldsymbol{dQ}_i^{(0)} = \boldsymbol{0} \in \mathbb{R}^{B_q \times d}$

  for $j = 1, \ldots, T_k$ **do**

    从全局内存加载 $\boldsymbol{K}^{(j)}, \boldsymbol{V}^{(j)}$

    计算注意力分数分块 $\boldsymbol{S}_i^{(j)} = \frac{\boldsymbol{Q}_i (\boldsymbol{K}^{(j)})^\top}{\sqrt{d}} \in \mathbb{R}^{B_q \times B_k}$

    计算注意力概率 $\boldsymbol{P}_i^{(j)} = \exp\left(\boldsymbol{S}_i^{(j)} - L_i\right) \in \mathbb{R}^{B_q \times B_k}$

    计算 $\boldsymbol{dP}_i^{(j)} = \boldsymbol{dO}_i \left(\boldsymbol{V}^{(j)}\right)^\top \in \mathbb{R}^{B_q \times B_k}$

    计算 $\boldsymbol{dS}_i^{(j)} = \boldsymbol{P}_i^{(j)} \circ \left(\boldsymbol{dP}_i^{(j)} - D_i\right) \in \mathbb{R}^{B_q \times B_k}$

29. 计算 $\boldsymbol{dQ}_i^{(j)} = \boldsymbol{dQ}_i^{(j-1)} + \boldsymbol{dS}_i^{(j)} \boldsymbol{K}^{(j)} / \sqrt{d} \in \mathbb{R}^{B_q \times d}$
30. **end for**
31. 把 $\boldsymbol{dQ}_i^{(T_k)}$ 写入全局内存，作为 $\boldsymbol{dQ}$ 的第 i 个分块。
32. **end for**
33. 返回 $\boldsymbol{dQ}, \boldsymbol{dK}, \boldsymbol{dV}$。

5 分布式数据并行训练

在本作业的下一部分，我们将探索使用多 GPU 训练语言模型的方法，重点是数据并行（data parallelism）。我们先从 PyTorch 分布式通信的入门讲解开始。然后，我们将研究分布式数据并行训练的一种朴素实现，接着实现并基准测试各种提升通信效率的改进。

5.1 PyTorch 单节点分布式通信

我们先来看一个简单的 PyTorch 分布式应用，它的目标是生成四个随机整数张量并计算它们的和。

在下面的分布式情形中，我们将派生（spawn）四个工作进程，每个进程生成一个随机整数张量。为了跨工作进程对这些张量求和，我们将调用 all-reduce 集合通信（collective communication）操作，它会把每个进程上的原数据张量替换为 all-reduce 的结果（即总和）。

现在我们来看代码。

```
import os
import torch
import torch.distributed as dist
import torch.multiprocessing as mp

def setup(rank, world_size):
    os.environ["MASTER_ADDR"] = "localhost"
    os.environ["MASTER_PORT"] = "29500"
    dist.init_process_group("gloo", rank=rank, world_size=world_size)

def distributed_demo(rank, world_size):
    setup(rank, world_size)
    data = torch.randint(0, 10, (3,))
    print(f"rank {rank} data (before all-reduce): {data}")
    dist.all_reduce(data, async_op=False)
    print(f"rank {rank} data (after all-reduce): {data}")

if __name__ == "__main__":
    world_size = 4
    mp.spawn(fn=distributed_demo, args=(world_size, ), nprocs=world_size, join=True)
```

运行上面的脚本之后，我们得到如下输出。正如预期，每个工作进程最初持有不同的数据张量。all-reduce 操作把所有工作进程的张量求和之后，每个工作进程上的数据被就地修改为 all-reduce

的结果。[2]

```
$ uv run python distributed_hello_world.py
rank 3 data (before all-reduce): tensor([3, 7, 8])
rank 0 data (before all-reduce): tensor([4, 4, 7])
rank 2 data (before all-reduce): tensor([6, 0, 7])
rank 1 data (before all-reduce): tensor([9, 5, 3])
rank 1 data (after all-reduce): tensor([22, 16, 25])
rank 0 data (after all-reduce): tensor([22, 16, 25])
rank 3 data (after all-reduce): tensor([22, 16, 25])
rank 2 data (after all-reduce): tensor([22, 16, 25])
```

现在让我们回头仔细看看上面的脚本。命令 `mp.spawn` 会派生 `nprocs` 个进程，用提供的 `args` 运行 `fn` 。此外，`fn` 会以 `fn(rank, *args)` 的形式被调用，其中 `rank` 是工作进程的索引（介于 0 到 `nprocs` -1 之间的值）。因此，我们的 `distributed_demo` 函数必须接受这个整数 rank 作为它的第一个位置参数。另外，我们传入了 `world_size` ，即工作进程的总数。

每个工作进程都属于一个进程组（process group），进程组通过 `dist.init_process_group` 初始化。进程组表示多个将通过一个共享的 master 来协调与通信的工作进程。master 由其 IP 地址和端口定义，运行 rank 为 0 的进程的就是 master。像 all-reduce 这样的集合通信操作作用于进程组中的每个进程。

在这个例子中，我们用 `"gloo"` 后端初始化进程组，但还有其他后端可用。特别地，`"nccl"` 后端会使用 NVIDIA NCCL 集合通信库，对于 CUDA 张量，它的性能通常更好。不过，NCCL 只能用于有 GPU 的机器，而 Gloo 可以在只有 CPU 的机器上运行。分布式 GPU 训练你应当始终使用 NCCL，只有在没有 GPU 可用的本地开发中才使用 Gloo。我们在这个例子中使用 Gloo，是因为它让我们能够在只有 CPU 的机器上本地执行和开发。

运行多 GPU 作业时，务必确保不同的 rank 使用不同的 GPU。一种方法是在 setup 函数中调用 `torch.cuda.set_device(rank)` ，这样 `tensor.to("cuda")` 就会自动把它移动到指定的设备。或者，你也可以显式地创建每个 rank 各自的设备字符串（例如 `device = f"cuda:{rank}"` ），然后把该设备字符串用作任何数据移动的目标设备（例如 `tensor.to(f"cuda:{rank}")` ）。

术语

在作业的其余部分（以及你可能在网上看到的各类其他资料）中，你可能会在 PyTorch 分布式通信的语境下遇到以下术语。虽然本作业聚焦于单节点、多进程的分布式训练，但这些术语对理解一般意义上的分布式训练也很有用。图 3 给出了直观的示意。

node 网络上的一台机器。

worker 参与分布式训练的一个程序实例。在本作业中，每个 worker 只有一个进程，因此我们会交替使用 worker、进程（process）和工作进程（worker process）这几个词。不过，一个 worker

[2]如果你多次运行这个脚本，会注意到输出的打印顺序是不确定的。由于这个应用运行在分布式环境中，我们无法控制命令执行的确切顺序——我们唯一的保证是：在 all-reduce 操作完成之后，各个进程将持有按位一致（bitwise identical）的结果张量。

也可能使用多个进程（例如为了加载训练数据），所以这些术语在实践中并不总是等价的。

world size 进程组中 worker 的总数。

global rank 唯一标识进程组中某个 worker 的整数 ID（介于 0 到 `world_size` -1 之间）。例如，当 world size 为 2 时，一个进程的 global rank 为 0（master 进程），另一个进程的 rank 为 1。

local world size 当应用跨多个节点运行时，local world size 是在给定节点上本地运行的 worker 数量。例如，如果我们有一个应用在 2 个节点上各派生 4 个 worker，那么 world size 是 8，local world size 是 4。注意，在单节点上运行时，worker 的 local world size 等价于（全局）world size。

local rank 唯一标识机器上本地 worker 索引的整数 ID（介于 0 到 `local_world_size` -1 之间）。例如，如果我们有一个应用在 2 个节点上各派生 4 个进程，那么每个节点上会有 local rank 为 0、1、2、3 的 worker。注意，运行单节点多进程分布式应用时，进程的 local rank 等价于它的 global rank。

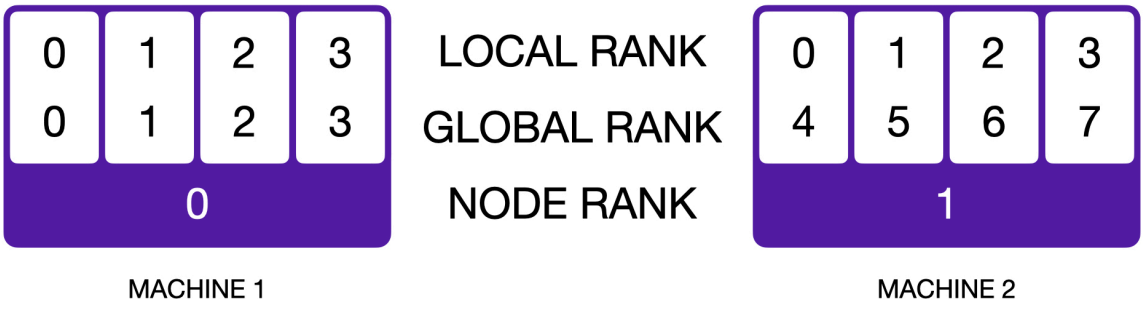


图 3: 运行在 2 个节点上、world size 为 8 的分布式应用示意图。每个工作进程由一个 global rank（0 到 7）和一个 local rank（0 到 3）标识。图取自 https://lightning.ai/docs/fabric/stable/advanced/distributed_communication.html

5.1.1 分布式应用基准测试的最佳实践

在作业的这一部分，你将对分布式应用进行基准测试，以更好地理解通信带来的开销。以下是一些最佳实践：

- 只要有可能，就在同一台机器上运行基准测试，以便进行受控的比较。
- 在对目标运算计时之前，先进行若干次预热（warm-up）。这对 NCCL 通信调用尤其重要。通常 5 次预热迭代就足够了。
- 在 GPU 上做基准测试时，调用 `torch.cuda.synchronize()` 等待 CUDA 操作完成。注意，即使调用带 `async_op=False` 的通信操作也需要这样做：该调用在操作被排入 GPU 队列时就会返回（而不是在通信实际完成时）。[3]

[3]更多细节见 https://github.com/pytorch/pytorch/issues/68112#issuecomment-965932386。

- 不同 rank 上的计时结果可能略有差异，因此通常要跨 rank 聚合测量结果，以改进估计。你会发现 all-gather 集合通信操作（具体来说是 `dist.all_gather_object` 函数）对收集所有 rank 的结果很有用。
- 一般而言，先在本地用 Gloo 在 CPU 上调试，然后按照题目要求在 GPU 上用 NCCL 做基准测试。

在后端之间切换应当很简单：只需修改你的 `init_process_group` 调用和张量设备转换即可。

问题（`distributed_communication_single_node`）：分布式通信（单节点）（5 分）

编写一个脚本，对单节点多进程设置下 all-reduce 操作的运行时间进行基准测试。上面的示例代码可以作为一个合理的起点。尝试改变以下设置进行实验：

all-reduce 数据大小 float32 数据张量，在 1MB、10MB、100MB、1GB 中取值。

GPU/进程数量 2、4 或 6。

资源要求：最多 6 块 GPU。每次基准测试运行耗时应少于 5 分钟。

交付物：比较多组设置的图和/或表格，并附上 2-3 句话，评述你的结果以及你对这些因素之间如何相互影响的思考。

5.2 分布式数据并行训练的朴素实现

在了解了用 PyTorch 编写分布式应用程序的基础知识之后，我们来构建一个分布式数据并行（distributed data parallel，DDP）训练的最小实现。

数据并行把批次拆分到多台设备（例如 GPU）上，使得在单台设备装不下的大批量大小上进行训练成为可能。例如，假如有四台设备，每台最多能处理批量大小 32，那么数据并行训练就能实现 128 的有效批量大小（effective batch size）。

下面是朴素地执行分布式数据并行训练的步骤。一开始，每台设备构建一个（随机初始化的）模型。我们使用 broadcast（广播）集合通信操作把模型参数从 rank 0 发送到所有其他 rank。训练开始时，每台设备都持有模型参数和优化器状态（例如 Adam 中累积的梯度统计量）的一份完全相同的副本。

1. 给定一个包含 n 个样本的批次，将该批次分片，每台设备得到 n/d 个互不重叠的样本（其中 d 是用于数据并行训练的设备数）。d 应当能整除 n，否则某些 rank 要做的工作会更多，整个步骤就会被最慢的 rank 卡成瓶颈。
2. 每台设备使用自己本地那份模型参数副本，对自己分到的 n/d 个样本运行一次前向传播和一次反向传播来计算梯度。注意，此时每台设备持有的梯度只是由它收到的 n/d 个样本计算出来的。

3. 然后我们使用 all-reduce 集合通信操作对不同设备上的梯度取平均，使每台设备都持有在全部 n 个样本上取平均后的梯度。

4. 接下来，每台设备执行一次优化器步骤来更新自己的参数副本——从优化器的视角看，它只是在优化一个本地模型。由于所有设备都从相同的初始模型和优化器状态出发，并且在每次迭代中使用相同的平均梯度，因此参数和优化器状态会在所有设备上保持同步。到这里，我们就完成了一次训练迭代，可以重复上述过程。

问题（`naive_ddp`）：朴素 DDP（5 分）

交付物：实现一种朴素形式的分布式数据并行训练：在反向传播结束后对所有参数梯度逐一进行 all-reduce。要测试你的实现，请先实现 `adapters.get_ddp` 和（可选的）`adapters.ddp_on_after_backward`，然后运行 `uv run pytest tests/test_ddp.py`。

问题（`naive_ddp_benchmarking`）：朴素 DDP 基准测试（3 分）

在这种朴素 DDP 实现中，每次反向传播结束后，参数梯度会在各个 rank 之间被逐一 all-reduce。为了更好地理解数据并行训练的开销，请编写一个脚本，用这种朴素的 DDP 实现对你之前实现的语言模型进行训练并做基准测试。测量每个训练步骤的总耗时，以及花在梯度通信上的时间占比。在第 2.1.2 节描述的 xl 模型规模下、单节点设置（1 节点 × 2 GPU）中收集测量数据。

交付物：对你的基准测试设置的描述，以及每种设置下测得的每个训练迭代的耗时和梯度通信耗时。

5.3 改进最小 DDP 实现

我们在第 5.2 节看到的最小 DDP 实现有两个关键局限：

1. 它为每个参数张量单独执行一次 all-reduce 操作。每次通信调用都会带来开销，因此把通信调用批量进行以尽量减少这种开销可能是有利的。

2. 它要等反向传播结束后才通信梯度。然而，反向传播是增量式计算的。因此，当某个参数的梯度一就绪，就可以立即把它通信出去，而不必等待其他参数的梯度。这让我们可以把梯度的通信与反向传播的计算重叠起来，从而降低分布式数据并行训练的开销。

在作业的这一部分，我们将依次解决这两个局限，并测量它们对训练速度的影响。

5.3.1 减少通信调用次数

与其为每个参数张量发起一次通信调用，不如看看能否通过把 all-reduce 批量化来提升性能。具体来说，我们会把想要 all-reduce 的梯度拼接（concatenate）成一个单独的张量，然后在所有 rank 之间对合并后的梯度做一次 all-reduce。使用 `torch._utils._flatten_dense_tensors` 和 `torch._utils._unflatten_dense_tensors` 可能会有帮助。

问题（`minimal_ddp_flat_benchmarking`）：最小 DDP 扁平化梯度基准测试（2 分）

修改你的最小 DDP 实现，改为通信一个包含所有参数展平（flatten）梯度的张量。在之前相同的条件下（1 节点 × 2 GPU，第 2.1.2 节描述的 xl 模型规模），将其性能与为每个参数张量各发起一次 all-reduce 的最小 DDP 实现进行比较。

交付物：在使用单个批量化 all-reduce 调用的分布式数据并行训练下，测得的每个训练迭代耗时与梯度通信耗时；并用 1-2 句话比较批量通信与逐一通信梯度的结果。

5.3.2 让计算与单个参数梯度的通信重叠

把通信调用批量化或许有助于降低发起大量小型 all-reduce 操作所产生的开销，但所有通信时间仍然直接构成开销。为了解决这个问题，我们可以利用这样一个观察：反向传播是逐层增量式计算梯度的（从损失出发、朝输入方向推进）——因此，一旦某个参数梯度就绪，我们就可以立即对它做 all-reduce，通过把反向传播的计算与梯度通信重叠，来降低数据并行训练的开销。

我们先实现并测试一个分布式数据并行包装器（wrapper）：在反向传播过程中，一旦某个参数张量就绪，就异步地对它进行 all-reduce。以下几点提示可能有帮助：

反向传播钩子（backward hooks） 要在反向传播中某个参数的梯度累积完成后自动对该参数调用一个函数，可以使用 `register_post_accumulate_grad_hook` 函数。[4]

异步通信 所有 PyTorch 集合通信操作都支持同步执行（`async_op=False`）和异步执行（`async_op=True`）。同步调用会阻塞，直到该集合操作在 GPU 上排入队列。这并不意味着 CUDA 操作已经完成，因为 CUDA 操作本身是异步的；尽管如此，之后使用该输出的函数调用仍会表现如预期。[5] 相反，异步调用会返回一个分布式请求句柄（request handle）——因此当函数返回时，该集合通信操作并不保证已经在 GPU 上排队，更不用说已经完成了。要等待操作在 GPU 上完成排队（从而其输出可以在后续操作中使用），可以对返回的通信句柄调用 `handle.wait()`。

例如，下面两个例子分别以同步和异步方式对一个张量列表中的每个张量做 all-reduce：

```
tensors = [torch.rand(5) for _ in range(10)]

# Synchronous, block until operation is queued on the GPU.
for tensor in tensors:
    dist.all_reduce(tensor, async_op=False)

# Asynchronous, return immediately after each call and
# wait on results at the end.
handles = []
for tensor in tensors:
    handle = dist.all_reduce(tensor, async_op=True)
    handles.append(handle)
```

[4] 更多信息和使用示例参见 https://pytorch.org/docs/stable/generated/torch.Tensor.register_post_accumulate_grad_hook.html。

[5] 在高级场景中，如果你使用多条 CUDA 流（stream），可能需要在流之间显式同步，以确保输出对后续操作就绪。参见 https://pytorch.org/docs/stable/notes/cuda.html#cuda-streams。

```
# ...
# Possibly execute other commands that don't rely on the all_reduce results
# ...

# Ensure that all-reduce calls were queued and
# therefore other operations depending on the
# all-reduce output can be queued.
for handle in handles:
    handle.wait()
handles.clear()
```

问题（ddp_overlap_individual_parameters ）：重叠单个参数通信的 DDP（5 分）

实现一个 Python 类来处理分布式数据并行训练。这个类应包装任意的 PyTorch nn.Module ，并负责在训练开始前广播权重（使所有 rank 拥有相同的初始参数），以及为梯度平均发起通信调用。我们推荐以下公共接口：

- def __init__(self, module: torch.nn.Module): ：给定一个已实例化、待并行化的 PyTorch nn.Module ，构造一个 DDP 容器，由它负责跨 rank 的梯度同步。
- def forward(self, *inputs, **kwargs): ：以给定的位置参数和关键字参数调用被包装模块的 forward() 方法。
- def finish_gradient_synchronization(self): ：被调用时，等待异步通信调用在 GPU 上完成。

要用这个类执行分布式训练，我们会把要包装的模块传给它，然后在运行 optimizer.step() 之前加入一次对 finish_gradient_synchronization() 的调用，以确保优化器步骤——一个依赖于梯度的操作——能够被安全地排入队列：

```
model = ToyModel().to(device)
ddp_model = DDP(model)

for _ in range(train_steps):
    x, y = get_batch()
    logits = ddp_model(x)
    loss = loss_fn(logits, y)
    loss.backward()
    ddp_model.finish_gradient_synchronization()
    optimizer.step()
```

交付物：实现一个容器类来处理分布式数据并行训练。该类应当把梯度通信与反向传播的计算重叠起来。要测试你的 DDP 类，请先实现适配器 adapters.get_ddp 和 adapters.ddp_on_after_backward （后者是可选的，取决于你的实现，可能用不上它）。

然后，运行 uv run pytest tests/test_ddp.py 执行测试。我们建议多次运行测试（例如 5 次），以确保它能可靠地通过。

问题（ddp_overlap_individual_parameters_benchmarking ）：重叠单个参数通信的 DDP 基准测试（1 分）

(a) 对你的 DDP 实现做基准测试：把反向传播计算与单个参数梯度的通信重叠起来。在与之前研究过的设置相同的条件下进行比较（最小 DDP 实现：或为每个参数张量各做一次 all-reduce，或对所有参数张量拼接后做单次 all-reduce），配置相同：1 节点、2 GPU，以及第 2.1.2 节描述的 xl 模型规模。

交付物：把反向传播与单个参数梯度通信重叠后测得的每个训练迭代耗时，并用 1-2 句话比较结果。

(b) 使用 Nsight 分析器为你的基准测试代码插桩（使用 1 节点、2 GPU、xl 模型规模的配置），比较最初的 DDP 实现与这个重叠实现。直观地比较两条轨迹（trace），并提供一张分析器截图，证明一种实现把计算与通信重叠了，而另一种没有。

交付物：2 张截图（一张来自最初的 DDP 实现，另一张来自这个把计算与通信重叠的 DDP 实现），直观地展示通信是否与反向传播发生了重叠。

6 优化器状态分片

分布式数据并行在概念上简单，而且往往非常有效，但它要求每个 rank 都持有模型参数和优化器状态各自的一份独立副本。这种冗余可能带来显著的内存开销。例如，AdamW 优化器会为每个参数维护两个浮点数，也就是说它消耗的内存是模型权重的两倍。S. Rajbhandari 等人 [5] 描述了几种在数据并行训练中减少这种冗余的方法：把 (1) 优化器状态、(2) 梯度和 (3) 参数划分到各个 rank 上，并在必要时在 worker 之间进行通信。

在作业的这一部分，我们将通过实现一个简化版的优化器状态分片（optimizer state sharding）来降低每个 rank 的内存消耗。每个 rank 的优化器实例不再保存所有参数的优化器状态，而只处理其中一个参数子集（约占 1/world_size ）。当每个 rank 的优化器执行优化器步骤时，它只更新自己这个分片内的那部分模型参数。随后，每个 rank 会把它更新过的参数广播给其他 rank，以确保模型参数在每次优化器步骤之后保持同步。

问题（optimizer_state_sharding ）：优化器状态分片（15 分）

实现一个 Python 类来处理优化器状态分片。该类应包装任意的输入 PyTorch optim.Optimizer ，并负责在每次优化器步骤之后同步更新过的参数。我们推荐以下公共接口：

- def __init__(self, params, optimizer_cls: Type[Optimizer], **kwargs: Any): ：初始化分片状态优化器。params 是待优化的参数集合（或参数组，以便用户能对模型的不同部分使用不同的超参数，例如不同的学习率）；这些参数将被分片到所有 rank 上。参数 optimizer_cls 指定要包装的优化器类型（例如 optim.AdamW ）。最后，其余的关键字参数都会被转发给 optimizer_cls 的构造函数。请确保在这个方法中调用 torch.optim.Optimizer 超类的构造函数。

- `def step(self, closure, **kwargs): ` ：以给定的 closure 和关键字参数调用被包装优化器的 `step()` 方法。更新参数之后，与其他 rank 同步。
- `def add_param_group(self, param_group: dict[str, Any]): ` ：该方法应向分片优化器添加一个参数组。它在分片优化器构造期间由超类构造函数调用，也可能在训练期间被调用（例如用于逐渐解冻模型的各层）。因此，这个方法应当负责在各 rank 之间分配模型的参数。

交付物：实现一个容器类来处理优化器状态分片。要测试你的分片优化器，请先实现适配器 `adapters.get_sharded_optimizer` 。然后，运行 `uv run pytest tests/test_sharded_optimizer.py` 执行测试。我们建议多次运行测试（例如 5 次），以确保它们能可靠地通过。

现在我们已经实现了优化器状态分片，接下来分析它对训练期间峰值内存占用的影响，以及它的运行时开销。

问题（`optimizer_state_sharding_accounting` ）：优化器状态分片核算（5 分）

(a) 编写一个脚本，对开启与不开启优化器状态分片时训练语言模型的峰值内存占用进行性能分析。使用标准配置（1 节点、2 GPU、xl 模型规模），报告模型初始化之后、优化器步骤之前一刻、以及优化器步骤之后一刻的峰值内存占用。结果与你的预期一致吗？请分解每种设置下的内存占用（例如参数占多少、优化器状态占多少，等等）。

交付物：用 2-3 句话给出峰值内存占用结果，并说明内存在模型与优化器各组件之间是如何划分的。

(b) 我们的优化器状态分片实现会如何影响训练速度？在标准配置（1 节点、2 GPU、xl 模型规模）下，测量开启与不开启优化器状态分片时每次迭代的耗时。

交付物：用 2-3 句话给出你的计时结果。

(c) 我们这种优化器状态分片的方法与 ZeRO stage 1（在 S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He [5] 中记作 ZeRO-DP P_{os}）有何不同？

交付物：用 2-3 句话总结二者的差异，尤其是与内存和通信量相关的差异。

7 完全分片数据并行

有了优化器状态分片和数据并行，我们已经能在数据并行这个维度上拆分优化器状态和激活值。然而，模型权重仍然是重复的——我们在每块 GPU 上都保存着所有权重的一份完整副本。

我们可以把数据并行（DP）维度变成完全分片数据并行（FSDP）维度来解决这个问题。使用 FSDP 时，每块 GPU 只保存每个权重张量属于自己的那一份切片，但在为前向或反向传播做准备时，需要通过 all-gather 从其他 GPU 拉取切片来拼出完整的权重张量。

为了避免 GPU 计算空等通信完成，大多数 FSDP 实现会提前调度该层的 all-gather，即在需要之前就把相关权重准备好，防止通信阻塞计算。这样，权重分片通信就不会出现在关键路径（critical path）上——也就是说，只要通信能跟上计算并被妥善调度，它就没有成本。

有些层在内存和计算上都足够小，为它们付出一次传输的延迟开销并不值得。你应当把这些层标记为不由 FSDP 分片。在我们的架构中，这主要是各个 norm 层的情形。这样一来，剩下的就是嵌入层（embedding layer）和每个线性层（linear layer）。

虽然必须以 FP32 存储主权重（master weights）（任何会被反复累积写入的值都对精度敏感），但权重并不需要以 FP32 来使用。在混合精度（mixed precision）下，我们总是在使用之前先把权重转换为低精度的计算数据类型，那么不妨在权重被通信之前就完成转换，以节省带宽。

问题（`fsdp`）：完全分片数据并行（15 分）

实现一个 Python 类用于完全分片数据并行训练。该类应包装任意的 PyTorch `nn.Module`（你的完整模型），并挂钩（hook into）或包装其中的每个 `Linear` 或 `Embedding` 层。我们推荐以下公共接口：

- `def __init__(self, module: torch.nn.Module, compute_dtype: torch.dtype | None = None): `：给定一个已实例化、待并行化的 PyTorch `nn.Module`，构造一个 FSDP 模块，由它负责权重的 all-gather 和梯度的 reduce-scatter。请确保你的钩子或模块包装器能及时地为前向传播 all-gather 好权重。为了限制内存使用，只有当当前层的前两层完成前向传播之后才开始收集。在反向传播中，你的钩子或模块包装器应当 all-gather，使权重可用于计算。当梯度可用时，应当把它们 reduce-scatter 到相应的 rank。注意在使用完之后释放收集来的权重。当提供了 `compute_dtype` 时，在通信或用于计算之前把权重转换为该 dtype，同时保持主权重和优化器更新为 FP32。
- `def forward(self, *inputs, **kwargs): `：以给定的位置参数和关键字参数调用被包装模块的 `forward()` 方法。
- `def finish_gradient_synchronization(self): `：被调用时，等待异步通信调用在 GPU 上完成。

交付物：实现一个容器类来处理完全分片数据并行训练。该容器的每个分片都应当与作业 1 的标准 AdamW 实现兼容。要测试你的 FSDP 实现，请实现适配器 `adapters.get_fsdp`。用 `uv run pytest tests/test_fsdp.py` 运行测试。我们建议多次运行测试（例如 5 次），以捕捉任何竞争条件（race condition）。

问题（`fsdp_accounting`）：FSDP 核算（5 分）

(a) 根据你在第 6 节中的分析，实现 FSDP 预计能从峰值中节省多少内存？在你的计算中，可以忽略为向每块 GPU all-gather 权重所需的预分配缓冲区的大小。

交付物：用 2-3 句话给出你的结论。

(b) 在两块 GPU 上对 xl 模型做性能分析，并留意权重的 all-gather。通信能否及时完成、赶上前向传播？

交付物：用 2-3 句话给出你的计时结果。请附上 Nsight 截图来支持你的结论。

8 分析并行策略

我们还可以沿更多维度来并行化训练过程。一些常见的策略包括：

- **数据并行（DP）**——数据批次被拆分到多台设备上，每台设备为自己的批次计算梯度，然后在各设备之间对梯度取平均。
- **完全分片数据并行（FSDP）**——在数据并行的基础上，再把优化器状态、梯度和权重拆分到各设备上以减少内存占用。于是设备在前向和反向传播期间需要从其他设备收集权重分片。
- **张量并行（TP）**——权重矩阵沿输入或输出维度分片。设备计算与自己分片对应的激活值，然后这些激活值再在设备之间进行归约（reduce）或收集（gather）。
- **流水线并行（PP）**——模型按层拆分为多个阶段（stage），每个阶段在不同设备上运行。
- **专家并行（EP）**——混合专家（Mixture-of-Experts）模型中的专家被拆分到不同设备上，每台设备计算自己那个专家的输出。

在本节中，我们将在一个简化的设定下做一些基础计算，以便在各并行策略之间做出选择，并决定如何组合它们。首先，我们关注 DP、FSDP、TP 及它们的组合。我们的方法是：计算每种策略的通信成本，并将其与计算成本进行比较，从而得出在通信成本成为瓶颈之前我们可以扩展到多少台设备。

关于 TPU/GPU 拓扑和并行策略更详细的讨论，TPU Scaling Book（J. Austin 等人 [6]）是一份极好的资料。关于流水线并行更详细的讨论，参见 The Ultra-Scale Playbook 附录（H. Z. P. N. M. M. L. W. T. W. Nouamane Tazi Ferdinand Mom [7]）。这几本书的其余部分也包含许多你可能用得上的其他信息。

8.1 通信原语

我们的第一步是理解通信原语。在简化的设定中，假设我们有 N 台设备，编号为 $0, \ldots, N-1$，每对设备之间都由一条链路相连。我们还假设每台设备拥有 W 的出口（egress，即向外发送）带宽；换句话说，每台设备能以每秒 W 字节的速率向另一台设备发送数据。我们会如何实现 gather 和 reduce 呢？

实现 all-gather 操作的一种常见方式是**环形 all-gather**（ring all-gather）。回顾一下，在 all-gather 中，每台设备 i 起初拥有一个大小为 S/N 的数据块（chunk）x_i，最终拥有完整的 $x = [x_0, \ldots, x_{N-1}]$（大小为 S 字节）。在环形 all-gather 中，我们把设备排成一个圆圈。每一步中，每台设备把它当前持有的块发送给右边下一台设备，并保存从左边设备收到的块。这个过程不断重复：每台设备把它刚收到的块传给右边，并从左边收到一个新块。经过 $N-1$ 步之后，每台设备都拥有了完整的张量。

在我们的理想化设定中，每台设备每步同时发送一个大小为 S/N 的块，出口带宽为 W，而总共有 $N-1$ 步，所以环形 all-gather 耗时

$$\frac{N-1}{N}\frac{S}{W} \tag{20}$$

秒。

接下来，我们分析**环形 reduce-scatter**（ring reduce-scatter）。在 reduce-scatter 中，每台设备 i 起初拥有大小为 S 的完整张量 $x^{(i)}$。我们想计算归约结果 $y = \sum_{i=0}^{N-1} x^{(i)}$，但要让每台设备 i 最终只拥有一个大小为 S/N 的块 y_i。与环形 all-gather 一样，我们先把设备排成一个圆圈。每台设备先把自己的张量 $x^{(i)}$ 划分成 N 个块 $[x_0^{(i)}, \ldots, x_{N-1}^{(i)}]$，每块大小为 S/N。然后我们像环形 all-gather 那样传递这些块，不同的是，在把块传递出去之前，每台设备会把自己的贡献加到这个块上（该块保存的是一个部分和）。具体来说：

对于第 $t = 1, \ldots, N-1$ 步，设备 i 执行以下操作：

- 若 $t = 1$，初始化 $y \leftarrow x^{(i)}$，它保存到目前为止的部分和；
- 把块 $y_{(i-t) \bmod N}$ 发送给设备 $(i+1) \bmod N$；
- 从设备 $(i-1) \bmod N$ 接收块 $z_{(i-t-1) \bmod N}$；
- 更新你那份部分和的副本：$y_{(i-t-1) \bmod N} \leftarrow y_{(i-t-1) \bmod N} + z_{(i-t-1) \bmod N}$。

经过 $N-1$ 步之后，设备 i 就拥有了块 y_i 的完整和，因此环形 reduce-scatter 耗时

$$\frac{N-1}{N}\frac{S}{W} \tag{21}$$

秒，与环形 all-gather 相同。

最后来实现**环形 all-reduce**（ring all-reduce）。在 all-reduce 中，每台设备 i 起初拥有大小为 S 的完整张量 $x^{(i)}$，最终拥有归约结果 $y = \sum_{i=0}^{N-1} x^{(i)}$。我们把 all-reduce 实现为一次环形 reduce-scatter 接一次环形 all-gather，因此环形 all-reduce 耗时

$$2\frac{N-1}{N}\frac{S}{W} \tag{22}$$

秒。

问题（`alternate_ring_all_reduce`）：另一种环形 all-reduce（1 分）

我们不把 all-reduce 实现为环形 reduce-scatter 接环形 all-gather，而是使用以下算法：

对于第 $t = 1, \ldots, N-1$ 步，设备 i 执行以下操作：

- 若 $t = 1$，初始化 $y \leftarrow x^{(i)}$，它保存到目前为止的部分和；
- 把 $x^{((i-t+1) \bmod N)}$ 发送给设备 $(i+1) \bmod N$；
- 从设备 $(i-1) \bmod N$ 接收 $x^{((i-t) \bmod N)}$；
- 更新你那份部分和的副本：$y \leftarrow y + x^{((i-t) \bmod N)}$。

在与上面相同的设定下（每台设备出口带宽为 W，每个 $x^{(i)}$ 的大小为 S），这个算法需要多长时间？

交付物：用 S、N 和 W 表示的答案，以及一句话的理由说明。

8.2 分析数据并行

有了通信原语，我们就可以着手分析并行策略了。我们来分析单个 FFN 层的并行化。回顾一下，给定输入 $\boldsymbol{x}$，我们的前向传播由下面各式给出：

$$\boldsymbol{x}_1 = \boldsymbol{x}\boldsymbol{W}_1 \tag{20}$$

$$\boldsymbol{x}_2 = \boldsymbol{x}\boldsymbol{W}_2 \tag{21}$$

$$\boldsymbol{z} = f(\boldsymbol{x}_1) * \boldsymbol{x}_2 \tag{22}$$

$$\boldsymbol{y} = \boldsymbol{z}\boldsymbol{W}_3, \tag{23}$$

其中 $\boldsymbol{x}$ 的形状为 (B, D)，$\boldsymbol{W}_1$ 和 $\boldsymbol{W}_2$ 的形状为 (D, D_{FF})，$\boldsymbol{W}_3$ 的形状为 (D_{FF}, D)。f 是我们的逐元素激活函数（例如 SiLU），$*$ 表示逐元素乘法。

把反向传播显式写出来也很有用。回顾一下，给定形状为 (B, D) 的 $\boldsymbol{dy}$，反向传播由下面各式给出：

$$\boldsymbol{dz} = \boldsymbol{dy}\,\boldsymbol{W}_3^\top \tag{24}$$

$$\boldsymbol{dx}_2 = \boldsymbol{dz} * f(\boldsymbol{x}_1) \tag{25}$$

$$\boldsymbol{dx}_1 = \boldsymbol{dz} * f'(\boldsymbol{x}_1) * \boldsymbol{x}_2 \tag{26}$$

$$\boldsymbol{dx} = \boldsymbol{dx}_1\boldsymbol{W}_1^\top + \boldsymbol{dx}_2\boldsymbol{W}_2^\top \tag{27}$$

$$\boldsymbol{dW}_3 = \boldsymbol{z}^\top \boldsymbol{dy} \tag{28}$$

$$\boldsymbol{dW}_2 = \boldsymbol{x}^\top \boldsymbol{dx}_2 \tag{29}$$

$$\boldsymbol{dW}_1 = \boldsymbol{x}^\top \boldsymbol{dx}_1, \tag{30}$$

其中 $*$ 表示逐元素乘法。

回顾一下，在使用 N_{DP} 台设备的数据并行中，我们把输入 $\boldsymbol{x}$ 分片成大小为 $\left(\frac{B}{N_{\text{DP}}}, D\right)$ 的分片 $\boldsymbol{x}^{(i)}$。前向传播照常进行、不需要任何集合通信，产生大小为 $\left(\frac{B}{N_{\text{DP}}}, D\right)$ 的激活值 $\boldsymbol{y}^{(i)}$。在反向传播中，按批分片的激活值照常参与计算，设备 i 最终得到的梯度为

$$\boldsymbol{dW}_3^{(i)} = \left(\boldsymbol{z}^{(i)}\right)^\top \boldsymbol{dy}^{(i)} \tag{31}$$

$$\boldsymbol{dW}_2^{(i)} = \left(\boldsymbol{x}^{(i)}\right)^\top \boldsymbol{dx}_2^{(i)} \tag{32}$$

$$\boldsymbol{dW}_1^{(i)} = \left(\boldsymbol{x}^{(i)}\right)^\top \boldsymbol{dx}_1^{(i)}, \tag{33}$$

其中，我们不再对所有 B 个外积求和，而只对属于我们这份输入分片的 $\frac{B}{N_{\text{DP}}}$ 个外积求部分和。然后，我们需要跨设备做一次 all-reduce，才能得到完整的梯度 $\boldsymbol{dW}_3$、$\boldsymbol{dW}_2$ 和 $\boldsymbol{dW}_1$。

问题（data_parallel_calcs ）：数据并行计算（3 分）

现在我们拥有了计算数据并行何时会被通信卡成瓶颈所需的一切。设 C（单位为 FLOP/s）表示设备加速器的速度，W（单位为字节/秒）表示每台设备的出口带宽。于是我们可以分别计算计算时间与通信时间。由于计算和通信可以重叠，当通信时间超过计算时间时，我们就遇到了瓶颈。我们假设所有权重和激活值都是 FP16（即两个字节）。

(a) 在 N_{DP} 路数据并行下，计算反向传播需要多少 FLOPs？你可以忽略所有非矩阵乘法（non-matmul）运算。回顾一下，一个 $(A,B)(B,C)\to(A,C)$ 的矩阵乘法需要 $2ABC$ flops。

交付物：用 B、D、D_{FF} 和 N_{DP} 表示的答案，以及一句话的理由说明。

(b) 在 N_{DP} 路数据并行下，反向传播需要多少通信时间？

交付物：用 B、D、D_{FF}、N_{DP} 和 W 中的若干个表示的答案，以及一句话的理由说明。

(c) 固定其他参数，N_{DP} 最大可以到多少，才会遇到通信瓶颈？

交付物：一个不等式：一边为 N_{DP}，另一边为用 B、D、D_{FF}、C 和 W 中的若干个表示的表达式，并附一句话的理由说明。

8.3 分析完全分片数据并行

接下来分析 FSDP。回顾一下，与 DP 一样，FSDP 也会对输入和激活值的批量维度做分片。此外，为了节省内存，我们还会分片优化器状态、梯度和权重。我们可以沿任意一个维度分片权重，从而在设备 i 上得到分片 $\boldsymbol{W}_1^{(i)}$、$\boldsymbol{W}_2^{(i)}$ 和 $\boldsymbol{W}_3^{(i)}$，每个大小为 $\frac{D\,D_{\mathrm{FF}}}{N_{\mathrm{FSDP}}}$。

在前向传播中，我们只需在按批分片的输入上执行数据并行的前向传播即可。但为此，回顾一下，我们需要先跨设备 all-gather 权重分片：

$$\boldsymbol{W}_1 = \text{all-gather}\left(\left\{\boldsymbol{W}_1^{(i)}\right\}_{i=0}^{N_{\mathrm{FSDP}}-1}\right) \tag{34}$$

$$\boldsymbol{W}_2 = \text{all-gather}\left(\left\{\boldsymbol{W}_2^{(i)}\right\}_{i=0}^{N_{\mathrm{FSDP}}-1}\right) \tag{35}$$

$$\boldsymbol{W}_3 = \text{all-gather}\left(\left\{\boldsymbol{W}_3^{(i)}\right\}_{i=0}^{N_{\mathrm{FSDP}}-1}\right) \tag{36}$$

（执行按批分片的前向传播） (37)

注意，在什么时候做 all-gather 上我们有一定的自由度：只要在权重被使用之前完成 all-gather 即可，并且随后应当把它丢弃，以保持较低的内存开销。为简单起见，本节中我们把三个 all-gather 列在一起。

与前向传播一样，在反向传播中我们需要先跨设备 all-gather 权重分片。然后就可以执行数据并行的反向传播，只是不再需要做 all-reduce，因为每台设备只需要它自己那个分片的梯度。因此，我们改做 reduce-scatter：

$$\boldsymbol{W}_1 = \text{all-gather}\left(\left\{\boldsymbol{W}_1^{(i)}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{38}$$

$$\boldsymbol{W}_2 = \text{all-gather}\left(\left\{\boldsymbol{W}_2^{(i)}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{39}$$

$$\boldsymbol{W}_3 = \text{all-gather}\left(\left\{\boldsymbol{W}_3^{(i)}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{40}$$

（执行按批分片的反向传播） (41)

$$\boldsymbol{dW}_1^{(i)} = \text{reduce-scatter}\left(\left\{\boldsymbol{dW}_1^{(i)}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{42}$$

$$\boldsymbol{dW}_2^{(i)} = \text{reduce-scatter}\left(\left\{\boldsymbol{dW}_2^{(i)}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{43}$$

$$\boldsymbol{dW}_3^{(i)} = \text{reduce-scatter}\left(\left\{\boldsymbol{dW}_3^{(i)}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{44}$$

注意，这里的分片记号（上标 (i)）被重载（overload）了：reduce-scatter 的输入是完整权重上的部分和，而输出是分片权重上的完整和。

问题（`fsdp_calcs`）：完全分片数据并行计算（3 分）

在与数据并行计算相同的设定下，我们来计算 FSDP 何时会遇到通信瓶颈。

(a) 在 N_{FSDP} 路 FSDP 下，计算反向传播需要多少 FLOPs？前向传播呢？

交付物：两个分别用 B、D、D_{FF} 和 N_{FSDP} 表示的答案，并各附一句话的理由说明。

(b) 在 N_{FSDP} 路 FSDP 下，反向传播需要多少通信时间？前向传播呢？

交付物：两个分别用 B、D、D_{FF}、N_{FSDP} 和 W 中的若干个表示的答案，并各附一句话的理由说明。

(c) 固定其他参数，N_{FSDP} 最大可以到多少，反向传播才会遇到通信瓶颈？前向传播呢？

交付物：两个不等式：一边为 N_{FSDP}，另一边为用 B、D、D_{FF}、C 和 W 中的若干个表示的表达式，并各附一句话的理由说明。

8.4 分析张量并行

在实践中，FSDP 常常与一种叫做张量并行（tensor parallelism，TP）的并行策略结合使用。在 TP 中，我们把每个权重矩阵的输入维度或输出维度在设备间分片。输入维度分片通常称为“行并行”（row parallel），而输出维度分片通常称为“列并行”（column parallel）。

具体来说，假设我们想对矩阵乘法 $\boldsymbol{xW}$ 做分片，其中 $\boldsymbol{x}$ 的形状为 (B, D)，$\boldsymbol{W}$ 的形状为 (D, D_{FF})。在列并行中，我们拥有形状为 $\left(D, \frac{D_{\text{FF}}}{N_{\text{TP}}}\right)$ 的分片 $\boldsymbol{W}^{(i)}$，并且有

$$\boldsymbol{xW} = \text{all-gather}\left(\left\{\boldsymbol{xW}^{(i)}\right\}_{i=0}^{N_{\text{TP}}-1}\right). \tag{45}$$

另一方面，对于行并行，我们拥有形状为 $\left(\frac{D}{N_{\mathrm{TP}}}, D_{\mathrm{FF}}\right)$ 的分片 $\boldsymbol{W}^{(i)}$，并且每台设备还会先把输入 $\boldsymbol{x}$ 收缩（narrow）成形状为 $\left(B, \frac{D}{N_{\mathrm{TP}}}\right)$ 的切片 $\boldsymbol{x}^{(i)}$，然后再做矩阵乘法。于是我们有

$$\boldsymbol{x}\boldsymbol{W} = \text{all-reduce}\left(\left\{\boldsymbol{x}^{(i)}\boldsymbol{W}^{(i)}\right\}_{i=0}^{N_{\mathrm{TP}}-1}\right). \tag{46}$$

为了并行化我们的 FFN，我们将使用一种特定的张量并行配置：$\boldsymbol{W}_1$ 和 $\boldsymbol{W}_2$ 采用列并行（沿输出维度分片），而 $\boldsymbol{W}_3$ 采用行并行（沿输入维度分片）。由于行并行权重只需要输入的一个切片，这种配置让我们可以省去列并行权重之后的那次 all-gather。给定大小为 (B, D) 的输入 $\boldsymbol{x}$，该策略的前向传播如下：

$$\boldsymbol{x}_1^{(i)} = \boldsymbol{x}\boldsymbol{W}_1^{(i)} \tag{47}$$

$$\boldsymbol{x}_2^{(i)} = \boldsymbol{x}\boldsymbol{W}_2^{(i)} \tag{48}$$

$$\boldsymbol{z}^{(i)} = f\left(\boldsymbol{x}_1^{(i)}\right) * \boldsymbol{x}_2^{(i)} \tag{49}$$

$$\boldsymbol{y}^{(i)} = \boldsymbol{z}^{(i)}\boldsymbol{W}_3^{(i)} \tag{50}$$

$$\boldsymbol{y} = \text{all-reduce}\left(\left\{\boldsymbol{y}^{(i)}\right\}_{i=0}^{N_{\mathrm{TP}}-1}\right), \tag{51}$$

其中 $\boldsymbol{W}_1^{(i)}$ 和 $\boldsymbol{W}_2^{(i)}$ 的形状为 $\left(D, \frac{D_{\mathrm{FF}}}{N_{\mathrm{TP}}}\right)$，$\boldsymbol{W}_3^{(i)}$ 的形状为 $\left(\frac{D_{\mathrm{FF}}}{N_{\mathrm{TP}}}, D\right)$。

问题（`tp_calcs`）：张量并行计算（4 分）

在与 DP 和 FSDP 计算相同的设定下，我们来计算 TP 何时会遇到通信瓶颈。

(a) 给定大小为 (B, D) 的输入 $\boldsymbol{dy}$，写出上述张量并行策略的反向传播（其中 $\boldsymbol{W}_1^{(i)}$ 和 $\boldsymbol{W}_2^{(i)}$ 的形状为 $\left(D, \frac{D_{\mathrm{FF}}}{N_{\mathrm{TP}}}\right)$，$\boldsymbol{W}_3^{(i)}$ 的形状为 $\left(\frac{D_{\mathrm{FF}}}{N_{\mathrm{TP}}}, D\right)$）。

交付物：一系列描述反向传播的等式，用 $\boldsymbol{dy}$、分片权重（$\boldsymbol{W}_1^{(i)}$、$\boldsymbol{W}_2^{(i)}$、$\boldsymbol{W}_3^{(i)}$）、前向传播保存下来的激活值（$\boldsymbol{x}$、$\boldsymbol{x}_1^{(i)}$、$\boldsymbol{x}_2^{(i)}$、$\boldsymbol{z}^{(i)}$、$\boldsymbol{y}^{(i)}$）、通信原语以及你想定义的任何中间变量来表示。这些等式应当产生每台设备的梯度 $\boldsymbol{dW}_1^{(i)}$、$\boldsymbol{dW}_2^{(i)}$、$\boldsymbol{dW}_3^{(i)}$，以及反向传播的输出 $\boldsymbol{dx}$。你可以随意参考第 8.2 节中未分片的反向传播并加以修改。

(b) 在 N_{TP} 路 TP 下，计算前向传播需要多少 FLOPs？反向传播呢？

交付物：两个分别用 B、D、D_{FF} 和 N_{TP} 表示的答案，并各附一句话的理由说明。

(c) 在 N_{TP} 路 TP 下，前向传播需要多少通信时间？反向传播呢？

交付物：两个分别用 B、D、D_{FF}、N_{TP} 和 W 中的若干个表示的答案，并各附一句话的理由说明。

(d) 固定其他参数，N_{TP} 最大可以到多少，反向传播才会遇到通信瓶颈？前向传播呢？

交付物：两个不等式：一边为 N_{TP}，另一边为用 B、D、D_{FF}、C 和 W 中的若干个表示的表达式，并各附一句话的理由说明。

8.5 二维并行（FSDP + TP）

我们终于可以把各种并行策略组合起来了！在本节中，我们来看看如何最优地组合 FSDP 与 TP。提示：在前几节中，你应当已经发现，批量大小和模型规模这两个参数限制了你能扩展到的设备数量；而把一切都变大，你就能在不遇到通信瓶颈的情况下继续扩展设备。遗憾的是，批量大小超过某个临界点之后，继续放大会开始损害性能，因为梯度噪声显著缩小，失去了 SGD 的隐式正则化特性；这个临界点通常被称为“临界批量大小”（critical batch size）。而缩放定律（scaling law）通常会告诉我们想把模型做到多大（这将是你下一次作业中的任务！）。

在本节中，我们考虑一个简化的设定：有人带着所有问题参数（批量大小、模型规模、带宽、加速器速度）来找你。你的任务是选出一组 FSDP 与 TP 的配置，使其能扩展到尽可能多的设备，同时保持受计算限制（compute-bound）而不是受通信限制（communication-bound）。

我们先来过一遍把 FSDP 与 TP 组合起来的具体机制。每台设备将拥有一个 TP rank $i = 0, \ldots, N_{\text{TP}} - 1$ 和一个 FSDP rank $j = 0, \ldots, N_{\text{FSDP}} - 1$，构成一个共有 $N = N_{\text{TP}} N_{\text{FSDP}}$ 台设备的二维网格。按照 TP 的做法，我们先把 $\boldsymbol{W}_1$ 和 $\boldsymbol{W}_2$ 沿输出维度分片，把 $\boldsymbol{W}_3$ 沿输入维度分片。因此，我们将不得不在激活值上插入一个 TP 式的 all-reduce。接着，应用 FSDP：我们对输入的批量维度做分片，并把每个权重矩阵沿 TP 没有分片的那个维度进一步分片。然后，我们需要在做 TP 式的前向/反向传播之前，在权重上插入 FSDP 式的 all-gather，并在 TP 式的反向传播之后，对权重梯度做 reduce-scatter。

最终的结果是，每台设备 (i, j) 持有权重分片 $\boldsymbol{W}_1^{(i,j)}$ 和 $\boldsymbol{W}_2^{(i,j)}$（形状为 $\left(\frac{D}{N_{\text{FSDP}}}, \frac{D_{\text{FF}}}{N_{\text{TP}}}\right)$），以及 $\boldsymbol{W}_3^{(i,j)}$（形状为 $\left(\frac{D_{\text{FF}}}{N_{\text{TP}}}, \frac{D}{N_{\text{FSDP}}}\right)$）。给定大小为 $\left(\frac{B}{N_{\text{FSDP}}}, D\right)$ 的按批分片输入 $\boldsymbol{x}^{(j)}$，我们可以把前向传播写成：

$$\boldsymbol{W}_1^{(i)} = \text{all-gather}\left(\left\{\boldsymbol{W}_1^{(i,j)}\right\}_{j=0}^{N_{\text{FSDP}}-1}\right) \tag{52}$$

$$\boldsymbol{W}_2^{(i)} = \text{all-gather}\left(\left\{\boldsymbol{W}_2^{(i,j)}\right\}_{j=0}^{N_{\text{FSDP}}-1}\right) \tag{53}$$

$$\boldsymbol{W}_3^{(i)} = \text{all-gather}\left(\left\{\boldsymbol{W}_3^{(i,j)}\right\}_{j=0}^{N_{\text{FSDP}}-1}\right) \tag{54}$$

$$\boldsymbol{x}_1^{(i,j)} = \boldsymbol{x}^{(j)} \boldsymbol{W}_1^{(i)} \tag{55}$$

$$\boldsymbol{x}_2^{(i,j)} = \boldsymbol{x}^{(j)} \boldsymbol{W}_2^{(i)} \tag{56}$$

$$\boldsymbol{z}^{(i,j)} = f\left(\boldsymbol{x}_1^{(i,j)}\right) * \boldsymbol{x}_2^{(i,j)} \tag{57}$$

$$\boldsymbol{y}^{(i,j)} = \boldsymbol{z}^{(i,j)} \boldsymbol{W}_3^{(i)} \tag{58}$$

$$\boldsymbol{y}^{(j)} = \text{all-reduce}\left(\left\{\boldsymbol{y}^{(i,j)}\right\}_{i=0}^{N_{\text{TP}}-1}\right), \tag{59}$$

最终得到大小为 $\left(\frac{B}{N_{\text{FSDP}}}, D\right)$ 的按批分片输出 $\boldsymbol{y}^{(j)}$。为节省篇幅，本节省略反向传播，只关注前向传播。不过到这里，你已经拥有了把它自己写出来所需的全部信息。

问题（fsdp_tp_calcs ）：二维并行计算（6 分）

在与前面各处计算相同的设定下，我们来计算二维并行何时会遇到瓶颈。

(a) 在 N_{FSDP} 路 FSDP + N_{TP} 路 TP 下，计算前向传播需要多少 FLOPs？

交付物：用 B、D、D_{FF}、N_{FSDP} 和 N_{TP} 表示的答案，以及一句话的理由说明。

(b) 在 N_{FSDP} 路 FSDP + N_{TP} 路 TP 下，前向传播需要多少通信时间？假设沿每个轴的通信可以重叠（换句话说，沿 FSDP 轴的集合通信可以与沿 TP 轴的集合通信重叠）。

交付物：用 B、D、D_{FF}、N_{FSDP}、N_{TP} 和 W 中的若干个表示的答案，以及一句话的理由说明。提示：答案应当表示为两个量（FSDP 与 TP 的集合通信成本）之间的最大值（max），因为二者可以重叠。

(c) 在 N_{TP} 和 N_{FSDP} 的最优设定下，$N = N_{\text{TP}}N_{\text{FSDP}}$ 最大可以到多少，前向传播才会遇到通信瓶颈？

交付物：一个不等式：一边为 N，另一边为用 B、D、D_{FF}、C 和 W 中的若干个表示的表达式，并附几句话和等式作为理由说明。

(d) 现在假设 FSDP 轴与 TP 轴的集合通信无法重叠，因为它们共享相同的网络资源。在 N_{TP} 和 N_{FSDP} 的最优设定下，$N = N_{\text{TP}}N_{\text{FSDP}}$ 最大可以到多少，前向传播才会遇到通信瓶颈？不必担心把 N_{TP} 和 N_{FSDP} 截断为整数。

交付物：一个不等式：一边为 N，另一边为用 B、D、D_{FF}、C 和 W 中的若干个表示的表达式，并附几句话和等式作为理由说明。

9 排行榜

作业 2 的排行榜（leaderboard）将测试 8B 模型一次完整训练步骤的速度。我们向你发起挑战：给你的代码做基准测试，用你能想到的任何技巧来优化内存和运行时间。关键限制是：你不能改变模型的输入/输出行为。你的实现将针对 `cs336_basics` 目录中的模型进行测试。你的输入将以 BF16 加因果掩码（causal masking）进行测试，并且必须通过与常规实现相同的测试。实现还必须是你自己完成的，不能使用或复制已有的实现。你的计时应当在两块 B200 GPU 上、使用批量大小为 2、序列长度为 32,768 的样本进行测量。这个模型被有意设计得很难装进内存。完整配置见下面的代码。我们会对排名前 5-10 的提交做正确性与性能验证。我们用来给你的实现计时的测试如下：

```
class Config:
    ctx_len = 32768
    vocab_size = 151936
    d_model = 4096
    d_ff = 11008
    num_layers = 34
    num_heads = 32
    torch_dtype = torch.bfloat16
```

```
    is_causal = True
    batch_size = 2
cfg = Config()

def test_timing_forward_backward():
    labels, targets = torch.randint(high=cfg.vocab_size, size=(2, cfg.batch_size, cfg.ctx_len))

    model = BasicsTransformerLM(Config())
    optimizer = AdamW(model.parameters())

    def train_step():
        optimizer.zero_grad(set_to_none=True)
        res = model(labels)
        loss = cross_entropy(res, targets).sum()
        loss.backward()
        optimizer.step()

    timing_results = triton.testing.do_bench(train_step, rep=30_000, warmup=10_000)
    print(timing_results)
```

出于测试的目的，你可以把重复时间（repetition）和预热时间（warmup，以毫秒为单位）缩短一些。

以下是一些改进思路，以及确保模型装得下的办法：

- 调整核函数的分块（tile）大小（用 Triton autotune 来做！）
- 调整更多 Triton/`torch.compile` 配置参数
- 实现融合（fused）的 AdamW
- 基线实现会把完整的 logits（`[batch, seq_len, vocab_size]` ）完整地实例化出来。请编写一个把 LM head 与交叉熵损失融合在一起的核函数。你还可以让它以融合的方式顺带立即计算反向传播
- 改进 FlashAttention
  - 用 Triton 实现反向传播，而不只是用 `torch.compile` （见第 4.2.3 节）
  - 反向传播时对输入做两次遍历，一次算 dQ，另一次算 dK 和 dV，以避免块之间的原子操作（atomics）或同步
  - 做因果掩码时让程序实例提前终止，跳过那些必然全为零的分块
  - 把非掩码分块与分块对角线分开处理：计算前者时完全不需要比较索引，计算后者时只需一次比较
  - 在比 Hopper 更新的架构上使用 TMA（Tensor Memory Accelerator）功能，仿照我们教程中的类似模式
- 仅在确实需要时才使用激活检查点（activation checkpointing），用运行速度换取内存节省

问题（leaderboard ）：排行榜：最快的训练步骤（10 分）

基准测试将在两块 B200 GPU 上以批量大小 2 运行。你的提交将按一次完整训练步骤——前向传播、损失计算、反向传播和 AdamW 更新——的挂钟时间（wall-clock time）来评估。

在 PyTorch/Triton 缓存为空的情况下，你的基准测试运行必须在 10 分钟内完成，所以请当心过于激进的 `torch.compile` 和 Triton 自动调优。

交付物：使用 AdamW 的完整前向-反向训练步骤的最佳挂钟时间。

我们期待排行榜的提交成绩能击败 10 秒的朴素基线。

在此处把你的结果提交到排行榜：

github.com/stanford-cs336/assignment2-systems-leaderboard

参考文献

[1] T. Dao, “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning.” [Online]. Available: https://arxiv.org/abs/2307.08691

[2] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Re, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *Advances in Neural Information Processing Systems*, A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, Eds., 2022. [Online]. Available: https://openreview.net/forum?id=H4DqfPSibmx

[3] M. Milakov and N. Gimelshein, “Online normalizer calculation for softmax.” [Online]. Available: https://arxiv.org/abs/1805.02867

[4] H. He, “Making Deep Learning Go Brrrr From First Principles,” 2022, [Online]. Available: https://horace.io/brrr_intro.html

[5] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models.” 2020.

[6] J. Austin et al., “How to Scale Your Model,” 2025.

[7] H. Z. P. N. M. M. L. W. T. W. Nouamane Tazi Ferdinand Mom, “The Ultra-Scale Playbook: Training LLMs on GPU Clusters.” 2025.